MAY 2026

IBM Bob

A Public Developer's Guide

End-to-end coverage of IBM's agentic AI development partner — from install to enterprise rollout, with honest framing throughout.

FOR DEVELOPERS
CITIZEN DEVELOPERS
ENGINEERING LEADERS

Anil Singh

Based on publicly available information

16 PARTS · 7 APPENDICES · 109 PAGES

About this guide

This is an independent reference work on IBM Bob, compiled from publicly available sources current to May 2026: IBM's official product documentation at bob.ibm.com, the April 28, 2026 general-availability announcement and related IBM press materials, the IBM Bob YouTube channel and developer hub, public customer stories published by IBM, the Omdia Universe 2026 placement, and independent reviews and testing reports from IT Jungle, Northdoor, Planet Mainframe, MC Press, The Register, DevOps.com, and the IBM Champions and developer-blog community. Specific sources are credited inline throughout.

Disclaimer

This guide is not affiliated with, endorsed by, sponsored by, or otherwise approved by International Business Machines Corporation. It is an independent editorial work. All views, interpretations, evaluations, and any errors are the author's. Where IBM's marketing framing and independent reporting differ, both are presented and attributed.

Product details, pricing, capabilities, and roadmap items reflect publicly available information at the time of writing. Software products evolve quickly. Readers are encouraged to verify current specifics against IBM's official documentation at bob.ibm.com/docs before making procurement, architectural, or implementation decisions.

Trademarks

IBM, IBM Bob, BobShell, Bobcoin, Bobalytics, IBM Granite, IBM Z, IBM i, Watsonx, Watsonx Code Assistant, WCA, Instana, Maximo, QRadar, Guardium, IBMid, and related names and logos are trademarks or registered trademarks of International Business Machines Corporation in the United States and other countries. Anthropic and Claude are trademarks of Anthropic PBC. Mistral is a trademark of Mistral AI. Meta and Llama are trademarks of Meta Platforms, Inc. Microsoft, Visual Studio Code, and GitHub are trademarks of Microsoft Corporation. Cursor is a trademark of Anysphere Inc. All other product, service, and company names mentioned may be trademarks of their respective owners. All trademarks are used for descriptive and identification purposes only.

Quotation and attribution

Quotations from IBM publications, customer stories, analyst reports, and independent reviewers are used for purposes of commentary, criticism, and review under the fair-use provisions of US copyright law and analogous provisions in other jurisdictions. Sources are identified inline. Where extended attribution is appropriate, full citations appear in the surrounding prose.

© 2026 Anil Singh. All rights reserved.

May 2026 edition

How to read this guide

This guide exists because IBM Bob arrived as a different kind of artifact than the AI coding tools that came before it, and the public conversation has not yet caught up with what it actually is. Bob is not an autocomplete extension. It is not a chat window bolted onto VS Code. It is a standalone integrated development environment forked from VS Code, a separate terminal agent called BobShell, a multi-model routing layer that selects between Anthropic Claude, Mistral open-source models, IBM Granite small language models, and a set of fine-tuned specialists, and a governance fabric of approvals, sandboxes, ignore files, custom modes, custom rules, and checkpoints that wraps every action Bob takes. It is an opinion about how enterprise software development should work in 2026.

The audience is intentionally wide. The guide is written for the developer who needs to install Bob this afternoon and ship something by Friday, and for the engineering leader who needs to decide whether Bob belongs in the toolchain at all. The first half is evaluation framing: what Bob is, where it came from, how it positions against Claude Code, Cursor, Copilot, and Codex, what the early proof points look like, and where the honest gaps remain. The second half is hands-on: install, modes, tools, context, configuration, MCP, BobShell, security, pricing, and enterprise rollout, page by page. Sixteen parts in total, organized to be read sequentially or jumped into by topic. Seven appendices cover the reference material that doesn't fit cleanly into the main flow: the Galaxium Travels tutorial path, AGENTS.md and /init in depth, enterprise deployment specifics (firewall endpoints, Bobcoin packs, admin dashboard), the official troubleshooting reference, the canonical auto-approval risk matrix from IBM's security guidelines, an in-depth Bobcoin economics reference, and a complete glossary.

A few choices worth stating up front. Everything in this document comes from publicly available sources: IBM's product documentation at bob.ibm.com, the GA announcement from April 28 2026, the Think 2026 keynote coverage, the Omdia Universe placement, the IBM Bob YouTube channel, independent reviews from IT Jungle, Northdoor, Planet Mainframe, and Nick Litten, and customer stories published by IBM. Where IBM's marketing differs from independent reporting, both are presented. Where Bob has known weaknesses (the January 2026 security disclosure, the in-progress nature of iMode for IBM i with the Premium Package for i now scheduled for June 24, 2026, the honest caveats in IBM's own .bobignore documentation, and the dead-code patterns reported in independent reviews), those are surfaced rather than hidden. The goal is a guide a working developer would actually trust, not a brochure.

Read it cover to cover, or jump to the chapter that matches your need. The table of contents is the map.

Table of contents

Section	Chapter	Page
	About this guide	2
	How to read this guide	3
Part I	Why Bob exists and what makes it different	6
Part II	The Bob ecosystem at a glance	11
Part III	Hands-on: install, sign in, and your first hour	14
Part IV	Core concepts: tools, context, and the 200K window	18
Part V	Configuration: making Bob fit your team	22
Part VI	The fifteen features that define daily use	27
Part VII	The complete Bob commands reference	37
Part VIII	Designing custom Bob modes	42
Part IX	Learning Bob beyond the docs: YouTube and community	48
Part X	Use cases: where Bob earns its keep	51
Part XI	BobShell: agentic AI in the terminal	56
Part XII	Security, sandboxing, and the honest disclosures	60
Part XIII	Pricing, Bobcoins, and the enterprise plan	62
Part XIV	Bob for citizen developers and low-code practitioners	65
Part XV	Mainframe and platform specifics: IBM Z and IBM i	69
Part XVI	Independent perspectives and the competitive landscape	72
	Closing: what to do next	76
Appendix A	The Galaxium Travels tutorial path	78
Appendix B	AGENTS.md and /init in depth	81
Appendix C	Enterprise deployment reference	85

Section	Chapter	Page
Appendix D	Troubleshooting reference	88
Appendix E	The canonical auto-approval risk matrix	91
Appendix F	Bobcoin economics in depth	93
Appendix G	Glossary and acronym key	96
	A note on completeness	109

Part I: Why Bob exists and what makes it different

The thesis IBM is making

Every major AI coding tool released between 2023 and 2025 was, at heart, an answer to one question: how do we make developers type less code? GitHub Copilot answered with line-by-line autocomplete. Cursor answered with chat-driven file editing. Claude Code and OpenAI Codex answered with agentic terminals that could plan and execute multi-step work. They were all variations on the same theme. Less typing, faster output.

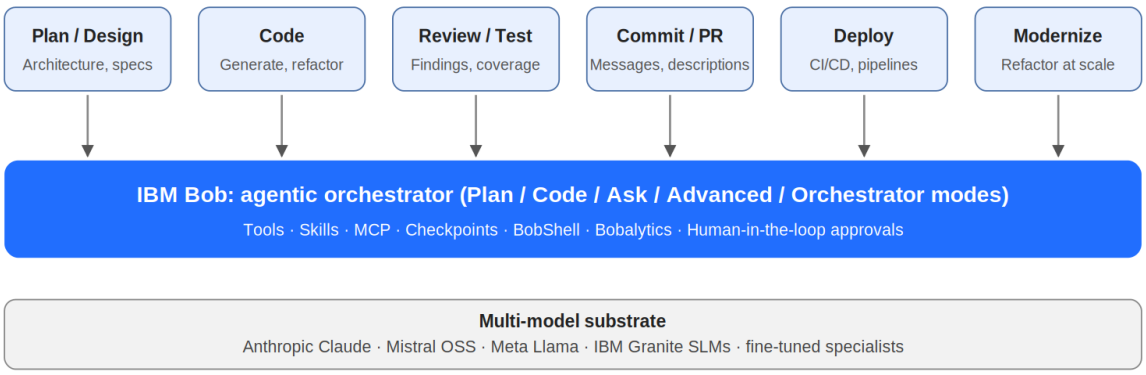
Bob is built on a different premise. IBM's position, made explicit by Neel Sundaresan in interviews around the April 2026 general-availability launch, is that the bottleneck in enterprise software development was never the typing. The bottleneck is everything else, understanding decades of accumulated complexity in legacy systems, coordinating change across teams that use different tools, satisfying compliance requirements, modernizing without breaking production, and reviewing AI-generated work before it ships. A faster coder who skips those steps is just a faster way to put bad code into production.

> *"Every business is racing to modernize. But speed without control and transparency is a liability. IBM Bob is how enterprises can move at AI speed without sacrificing the governance and security needs their businesses require."*
> — Dinesh Nirmal, Senior Vice President, IBM Software

So Bob was designed to operate across the full software development lifecycle rather than at one stage of it: planning, coding, reviewing, testing, committing, and modernizing, all coordinated by an agentic orchestrator that knows when to delegate to a fast model, when to delegate to a capable model, and when to stop and ask the developer for permission. The official IBM positioning calls this "AI-first SDLC" and the marketing slogan that captures it is "automate the mundane, augment the complicated."

Whether that thesis lands depends on whether the SDLC framing actually matters more than raw coding speed, and the early evidence is more interesting than the marketing copy. The Omdia Universe 2026 report on AI-assisted software development expanded its evaluation criteria for the first time to weight full-lifecycle capability rather than coding alone, and Bob was named a Leader. The shift in Omdia's criteria mirrors the shift in IBM's thesis, both are essentially arguments that the next phase of this market belongs to tools that handle the whole pipeline, not just the editor.

Where Bob operates: orchestration across the SDLC, not a single stage



The numbers IBM is putting on the table

Bob's internal pilot ran inside IBM from June 2025 through general availability in April 2026, growing from 100 developers to more than 80,000. Surveyed users reported an average productivity gain of 45 percent across modernization, security, and new development work. On specific teams the gains were sharper. The IBM Instana observability team reported a 70 percent reduction in time spent on selected tasks, averaging ten hours saved per developer per week. The Maximo asset-management team tested Bob on code generation and refactoring work that normally took days and completed it in hours, an estimated 69 percent time saving.

External clients have produced the harder-to-dismiss numbers, as published in IBM's customer case studies accompanying the GA launch. Blue Pearl, a cloud-services firm, used Bob to upgrade a Java application from a configuration that would have taken thirty engineering days to one that took three, a roughly 90 percent reduction in delivery time, with zero defects post-deployment and over 160 engineering hours saved through automated refactoring. APIS IT, which runs mission-critical government infrastructure in Croatia, used Bob to migrate complex .NET services in hours rather than weeks, documented decades-old JCL and PL/I systems with 100 percent operator-verified accuracy, and produced architecture analysis ten times faster than their previous baseline. Ernst & Young is using Bob to accelerate modernization of its global tax platform, with Christopher Aiken, Tax Platforms Leader and Chief Product Officer at EY, quoted in the launch materials citing the platform's ability to interpret deeply embedded business logic as the differentiator. The published Novacomp case study added a fourth dimension: a structural upgrade of a layered Java monolith to microservices completed in two days by a single engineer, work that would have taken a three-to-five-person team a couple months. All figures in this paragraph are from IBM's published customer stories and have not been independently audited; that caveat is revisited in Part XVI.

IBM has been transparent that some of these numbers come from self-reporting, and skeptical journalism has flagged that 80,000 internal users do not constitute an unbiased focus group. Those caveats are real and the guide returns to them later in the section on independent perspectives. But the breadth of the proof points (across modernization, new development, and

security testing; across mainframe and cloud; across IBM internal use and external customers) is unusually wide for a product less than a year out of private beta.

Where Bob came from

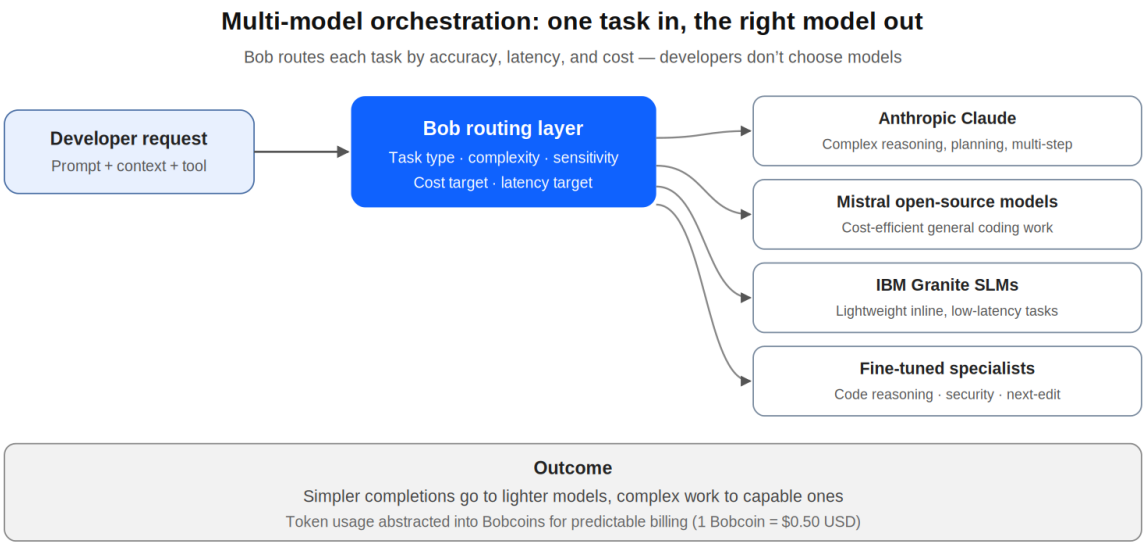
Bob is not a clean-sheet design. It is the rebrand and architectural rebuild of IBM watsonx Code Assistant, which itself existed in three product variants, WCA (the general developer assistant), WCA for Z (mainframe), and WCA for i (IBM i / RPG). Existing WCA clients have a documented adoption path to Bob and continue to be supported during the transition. The Bob name was introduced as "Project Bob" in private beta in October 2025, became generally available on April 28, 2026, and was launched alongside an early-access preview of the IBM Bob Premium Package for Z that brings WCA for Z's mainframe-specific capabilities into the Bob runtime. The IBM Bob Premium Package for i is scheduled to ship on June 24, 2026, completing the trilogy.

The change is not cosmetic. The WCA generation was delivered primarily as IDE extensions on top of VS Code and other editors. Bob is a full fork of VS Code, which means IBM controls the entire editor: file handling, command palette, terminal, settings, the lot. According to the IT Jungle review by IBM Champion Gregory Simmons, who tested an early-access build, the fork approach was explicitly chosen because IBM had hit limits maintaining a complex extension on top of someone else's editor with WCA for i. The trade-off is straightforward. A fork is more work to maintain but gives you complete control of the developer experience. An extension is cheaper to ship but always living inside someone else's product.

Two consequences follow. First, Bob-IDE offers to import your existing VS Code settings and extensions on first launch, and also offers import options for Cursor and Windsurf, signaling that IBM is targeting developers already using AI-first editors. Second, the Bob-IDE behaves more like a coherent product than an extension. Features like literate coding, checkpoints, the Bob Findings panel, and the auto-approve toolbar would be much harder to deliver as an extension that has to coexist with the host editor's own state management.

The multi-model thesis

The single most distinctive technical choice in Bob is that developers do not choose a model. Every task is routed by Bob to the model the routing layer thinks is most appropriate for that task, based on accuracy needs, latency needs, and cost. The available models include Anthropic Claude, open-source Mistral models, Meta Llama, IBM's own Granite family of small language models, and a set of fine-tuned specialists for code reasoning, security analysis, and next-edit prediction.



This is a deliberate move against what RedMonk senior industry analyst Kate Holterhoff has called "the paralysis of choice that comes from switching models between tasks." In tools like Cursor or the API consoles, developers often spend mental energy deciding which model to use for which job. Bob's argument is that developers should not have to think about this, the platform should make the choice and abstract the underlying cost into a single billing unit, the Bobcoin, that flattens token economics across models.

There are two genuine criticisms of this approach. The first, raised by Holterhoff herself, is that developers can be suspicious of black-box tools. When Bob picks a model for you, you do not know why, and if the result is wrong you do not know whether to blame the routing decision or the underlying model. IBM's response is partial, the Bobalytics dashboard surfaces model usage patterns and Bobcoin consumption, but it does not expose the routing logic itself. The second criticism, raised in The Register's launch coverage, is that GitHub recently had to rework its Copilot pricing because complex multi-step prompts were costing more in inference than the subscription brought in. Bob's whole reason for existing is multi-step agentic work, which means it faces the same economic problem from a steeper starting point. The Bobcoin overage system is IBM's answer, and it transparently pushes cost back to the user as work gets harder.

How Bob compares to the alternatives

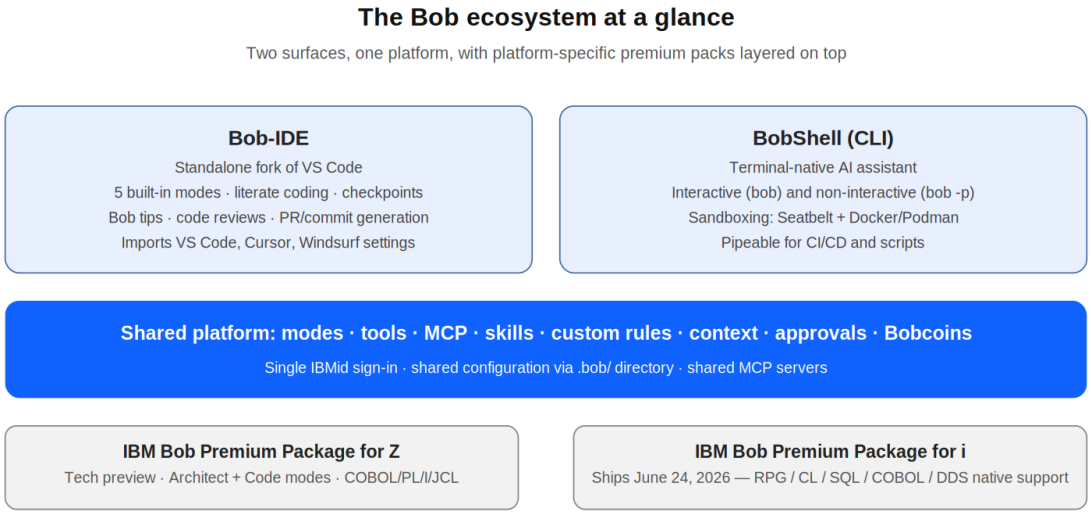
It is worth being precise about how Bob lines up against the tools developers are already using. Bob is closest in spirit to Claude Code and OpenAI Codex, agentic SDLC tools that orchestrate multi-step work, and most distinct from Copilot, which still leads with autocomplete. Cursor sits in between, with strong agent capabilities layered into a Cursor-branded fork of VS Code. The fact that Bob is itself a fork of VS Code with import paths for Cursor and Windsurf settings is not subtle: IBM is making it easy for developers who have already abandoned vanilla VS Code to try Bob without losing their setup.

A few comparison points worth holding in mind as you read the rest of this guide. Bob is the only one of the major AI coding tools that routes across multiple model providers transparently. Claude Code runs on Claude. Codex runs on OpenAI models. Copilot runs on a mix that's selected per-task but does not publish the routing logic. Cursor lets the user pick. Bob picks for you and tells you afterward what was used. Whether that's a feature or a limitation depends on your role: developers who want control will dislike it; engineering leaders worried about consistent enterprise costs will love it.

Bob is also the only one of these tools that ships with a built-in sandboxing layer at the agent level. BobShell supports both macOS Seatbelt and Docker/Podman sandboxing out of the box, with five distinct Seatbelt profiles spanning network-allowed-write-restricted through fully isolated. Claude Code recently added some sandboxing primitives but does not match Bob's profile depth. This is the security-and-governance story IBM keeps telling, and on the CLI side at least, the documentation backs it up.

Part II: The Bob ecosystem at a glance

Bob is not one thing. It is a coordinated set of surfaces and services that share a configuration, a model routing layer, a billing unit, and a sign-in. Before diving into install and daily workflow, it helps to see the map of what's actually in the box.



Bob-IDE: the standalone editor

Bob-IDE is the surface most developers will spend most of their time in. It is a full fork of Microsoft's Visual Studio Code with IBM modifications baked into the editor itself rather than layered on top as an extension. You install it as a regular application, a .pkg on macOS, a .exe on Windows, .deb or .rpm on Linux, and it runs as its own process. It is not an extension to your existing VS Code; it lives alongside it.

Because it's a fork rather than an extension, Bob-IDE can do things an extension cannot. The chat panel is a first-class part of the editor, not a sidebar webview competing for attention with other extensions. The auto-approve toolbar lives at the bottom of the chat panel and controls every tool Bob can run. The Bob Findings panel is a peer of the Problems panel rather than a tab inside it. The Bobcoin gauge in the top-right of the chat panel gives a real-time view of token-equivalent usage. Literate coding mode toggles the editor itself into a different state. None of this would be possible as an extension without significant friction.

On first launch Bob-IDE offers to import settings and extensions from VS Code, Cursor, and Windsurf. This is a deliberate competitive move, IBM is making it as low-friction as possible for developers who have already moved off vanilla VS Code to give Bob a try without rebuilding their environment.

BobShell: the terminal agent

BobShell is a separate command-line tool, installed independently of Bob-IDE, that brings Bob's capabilities into the terminal. It supports two modes of operation. The interactive mode (bob)

opens a conversational session in your terminal, type a question, get an answer, approve tool calls as they come up. The non-interactive mode (bob -p "your prompt") executes a one-shot prompt and returns the result, suitable for scripts and CI/CD pipelines.

BobShell is the surface where IBM has invested most heavily in sandboxing. By default, non-interactive mode runs in read-only mode, Bob can read files and analyze code but cannot modify anything until you add the --yolo flag, and even then write access is scoped to the directory where Bob was started. The --sandbox or -s flag enables full sandboxing via macOS Seatbelt or Docker/Podman, with five Seatbelt profiles ranging from permissive (network allowed, write restricted to project) to restrictive-closed (no network, maximum isolation). The same controls are available through the BOB_SHELL_SANDBOX environment variable or a settings.json file.

BobShell shares configuration with Bob-IDE through the .bob/ directory in your project and through global config files. Custom modes you define in the IDE are available in the shell, and vice versa. MCP servers configured in one are visible in the other. From the developer's perspective they are two interfaces to the same agent.

The shared platform: modes, tools, MCP, skills

Both surfaces, Bob-IDE and BobShell, sit on top of a shared platform. The platform exposes five built-in operating modes (Code, Ask, Plan, Advanced, Orchestrator), a set of agent tools (read, edit, browser, command, MCP), Model Context Protocol support for external integrations, a skills system that lets you define reusable workflows as markdown files with YAML frontmatter, a custom-rules system for codifying team standards, and a context-management layer with a 200,000-token window. Bobcoin metering applies across both surfaces, a single budget covers all your Bob usage regardless of where you invoke it from.

This shared platform is what makes Bob feel coherent rather than like two separate products. A skill you define in .bob/skills/code-review/ is automatically available in BobShell. A custom rule you put in .bobrules is honored by both. An MCP server registered in your project's .bob/mcp.json is connected to whether you talk to Bob through the IDE chat or the shell.

Premium packages: Z today, i next month

On top of the base platform, IBM has begun layering platform-specific premium packages. The first is the IBM Bob Premium Package for Z, announced alongside GA in April 2026 as a no-cost private technical preview. It integrates and enhances the capabilities of IBM watsonx Code Assistant for Z to deliver advanced features for enterprise-scale mainframe applications. It introduces two purpose-built mainframe modes, Architect mode for system-level reasoning across mainframe applications and Code mode for generating, refactoring, and transforming COBOL, PL/I, and JCL with Z-aware context. The Z package is delivered as an extension on top of the base Bob trial.

The IBM i story has just gotten concrete. The IBM Bob Premium Package for i is scheduled to ship on June 24, 2026, according to TechChannel's late-May 2026 reporting. The package will introduce IBM i Developer Mode, a persona specially configured for IBM i development, focused

on native IBM i languages including RPG, CL, SQL/DB2 for i, COBOL, and DDS. It will ship with approximately 40 curated skills for IBM i-specific use cases: documentation generation, fixed-to-free conversion, modernization workflows, and platform-specific best practices. Until that release, Bob can work with IBM i applications today through standard means, it understands RPG, CL, and SQL at the language level, but it cannot yet read source physical files or work directly with the IFS. The IT Jungle review by IBM Champion Gregory Simmons captures both the current state and the community's anticipation: Bob is a real tool for IBM i developers today, but the Premium Package for i is what would make it a category-defining one.

What you sign up for, in one paragraph

When you install Bob you get: a standalone VS Code-based IDE that imports your existing settings, a terminal agent with sandboxing, five built-in operating modes that constrain Bob's tool access, a 200,000-token context window with automatic condensation at 140K, a billing unit called the Bobcoin worth roughly fifty cents, a 30-day free trial with 40 Bobcoins, an opt-in (not opt-out) telemetry posture, integration with Model Context Protocol for external tool access, a custom-rules and custom-modes configuration model that mirrors team coding standards, a checkpoint system that snapshots your workspace before every modification using a shadow Git repository, and access to Anthropic Claude, Mistral OSS models, Meta Llama, IBM Granite SLMs, and a set of fine-tuned specialists under the hood, all routed automatically. That is the package.

Part III: Hands-on, install, sign in, and your first hour

The fastest way to understand Bob is to install it and complete the official quickstart, which walks through modernizing a Node.js application from version 16 to 22. This chapter takes you through everything from system requirements to the moment you've finished the quickstart and have a meaningful sense of what daily use feels like.

System requirements

Bob's published system requirements are modest. The IDE runs on macOS, Linux, and Windows. The official minimums are 4 GB of RAM (8 GB recommended) and 500 MB of available disk space. An active internet connection is required for both first-time authentication and ongoing model inference, Bob does not currently offer an offline mode. If you're behind a corporate firewall, IBM publishes guidance on the network endpoints that need to be reachable; for most developers this is not an issue but for some enterprise environments it requires coordination with IT.

Bobcoin economics depend on usage rather than hardware, so you do not need a beefy machine to run Bob effectively. The heavy lifting happens in IBM's cloud infrastructure where the routing layer and model inference run. Your machine handles the editor, the file system, and the terminal.

Installing Bob-IDE

Installation is platform-specific but follows the same pattern: download the installer from bob.ibm.com/download, run it, accept the defaults. On macOS, you download a .pkg file and complete the installation wizard. On Windows, you run an .exe installer that prompts for an install directory (the default is recommended). On Linux, packages are provided for Debian-based distributions and for Red Hat / Fedora, use your package manager to install.

macOS

```
# After downloading the .pkg from bob.ibm.com/download:
# Double-click the .pkg file, then follow the installer wizard.
# Installation requires admin password to write to /Applications.
```

Debian / Ubuntu

```
sudo dpkg -i ibm-bob_*.deb
sudo apt-get install -f   # resolve any dependency gaps
```

Red Hat / Fedora

```
sudo rpm -i ibm-bob-*.rpm
# Or use dnf:
sudo dnf install ./ibm-bob-*.rpm
```

Windows

```
# Run the downloaded .exe and follow the installation wizard.
# Default install location is recommended.
# Choose whether to add Bob to PATH if prompted.
```

First sign-in

On first launch, Bob will prompt you to sign in. Authentication uses your IBMid, if you don't have one, the sign-in screen links to IBM's free signup at ibm.com/account/reg. The flow opens your default browser, completes OAuth against IBM's identity provider, and returns to Bob with a session token. You'll only do this once per device unless you sign out explicitly.

After authentication, Bob offers to import settings from your existing development environment. The three options are Visual Studio Code, Cursor, and Windsurf. If you've been using any of these, picking the relevant option pulls in your themes, keybindings, and most extensions. If your extensions include things that conflict with Bob's built-in capabilities, say, GitHub Copilot or Cursor's chat panel, Bob will flag the conflicts and let you decide what to do.

> **First-launch settings worth setting immediately**
>
> Disable auto-approval. Bob's default is manual approval for every action. Keep it that way until you understand what each tool group does. The auto-approve toolbar lives just above the chat input, confirm it shows `Auto-approval disabled`.
>
> Enable telemetry only if you want to. Telemetry is opt-in, not opt-out, Bob does not collect usage data by default. If you want to help IBM improve the product, you can enable it under Settings → Application → Telemetry.
>
> Create a `.bobignore` file in any sensitive workspace. Even if it's empty for now, having the file in place means you can add patterns later without changing your habits.
>
> Run `/init` on your first real project. This generates a root AGENTS.md plus four mode-specific AGENTS-{mode}.md files in .bob/rules-{mode}/ directories, all of which Bob loads automatically as persistent project context. It is the single highest-leverage one-time investment per project. Appendix B covers the mechanics in depth.

The chat panel: components you'll use every day

The Bob chat panel opens on the right side of the editor by default. If it's not visible, click the Bob icon in the activity bar or use Cmd+L (Mac) / Ctrl+L (Windows/Linux). The panel has six elements you'll touch every session.

Element	What it does
Chat history	The running conversation, your prompts, Bob's responses, file edits, command outputs, and the inline approval buttons that appear when Bob proposes an action.

Element	What it does
Mode selector dropdown	Sits to the left of the chat input. Switches between Code, Ask, Plan, Advanced, and Orchestrator modes. Also accessible via /code, /ask, /plan, /advanced, /orchestrator slash commands.
Input field	Type your request in natural language. The @ symbol opens the context-mention picker; the / symbol opens the slash-command menu.
Enhance prompt button	The sparkles icon at the bottom right of the input field. Refines a vague prompt into a structured one with explicit requirements.
Auto-approve toolbar	Above the chat input. Hover to reveal toggles for each action group (Read, Write, Browser, MCP, Mode, Subtasks, Execute, Question, Todo).
Bobcoin gauge	Top-right of the panel. Shows current consumption against your monthly allocation. Hover for details.

The quickstart: modernizing a Node.js app from 16 to 22

IBM's official quickstart walks through upgrading a small TypeScript Express API from Node 16 to Node 22. It's the single fastest way to experience the full Bob workflow, mode switching, context mentions, approvals, Code mode for implementation, Ask mode for analysis, and literate coding for in-editor edits. The repository is public at github.com/IBM/bob-demo, and Docker is the only external prerequisite (it spares you from installing Node and dependencies locally).

The full walkthrough is in IBM's documentation; the short version follows here, with notes on what each step is actually teaching you.

1. Clone `https://github.com/IBM/bob-demo.git` through Bob's file picker. Bob asks you to trust the authors of the cloned folder before opening it, say yes.
2. Confirm auto-approval is disabled. Every Bob action in this tutorial should require explicit approval so you can see what tools Bob is calling.
3. In Code mode, ask Bob to create a Dockerfile for the existing Node 16 application. Bob will ask permission to read several files to understand the project structure, approve them. Bob will propose a Dockerfile; click `Save` to write it.
4. Ask Bob to build and test the legacy application using Docker. Bob will run the build, hit an error because the Dockerfile uses `npm ci` without a lockfile, recognize the error, propose a fix to use `npm install` instead, and re-run the build. This is the first taste of agentic loop behavior, Bob debugging its own work without being asked.
5. Switch to Ask mode (slash command `/ask`). Use @ context mentions to ask Bob what would need to change in package.json, tsconfig.json, and the Dockerfile to upgrade to Node 22. Ask mode is read-only, Bob can analyze but cannot modify.
6. Switch back to Code mode and ask Bob to perform the modernization, referencing all three files with @ mentions. Bob updates the Node version in package.json, bumps

TypeScript and @types/node, updates the tsconfig target from ES2020 to ES2022, and changes the Dockerfile base image.

7. Use literate coding to update a sample data file. Open `src/db.ts`, toggle literate coding with Cmd+M (Mac) / Ctrl+M (Windows/Linux), and write your instruction as a plain-language comment at the top of the file. Click Generate. Bob updates the relevant value and shows you a diff to accept or reject.
8. Ask Bob to verify the modernized app works by building and testing the Docker container. Bob runs the build, runs the tests, confirms the API responds, and cleans up the container, all in one approved sequence.

Total time end-to-end is under thirty minutes for a developer who has used an agentic AI tool before, and maybe sixty for someone who hasn't. By the end you will have used four of the five built-in modes, written code through both the chat panel and literate coding, watched Bob recover from its own errors, and produced a working modernized application. That is most of what daily use feels like.

What 'good' looks like in your first week

After the quickstart, the highest-leverage way to spend your first week with Bob is to use it on real work. The official best-practices documentation makes the point bluntly: "Bob learns best when helping with real problems rather than hypothetical scenarios." A few patterns to apply early.

- Start every complex task in Plan mode. Plan mode is read-only with markdown write access, which means Bob can create a planning document but cannot accidentally touch your code. Once the plan looks right, switch to Code mode and implement.
- Use context mentions aggressively. The single most common cause of bad Bob output is insufficient context. Prefer `@/src/utils/validation.ts:45-67` over `@/src/utils`, targeted is cheaper in Bobcoins and clearer in results.
- Commit often, in small atomic units, with Bob-generated commit messages. The sparkle icon next to the Source Control commit input box generates a message based on your staged changes. Atomic commits make checkpoint and /review work better.
- Treat the auto-approve toolbar like a dial, not a switch. Turn on Read auto-approval once you trust it, Bob reading files is genuinely low-risk. Keep Write and Execute on manual until you have a clear reason to change.
- When something goes wrong, start a new chat. Context poisoning is a real failure mode in long sessions; the documentation explicitly says *no* corrective prompt reliably fixes a poisoned conversation. Cmd+N / Ctrl+N for a new chat is your friend.

By the end of week one you should have working intuitions about which mode to use when, which context mentions cost most, when to let Bob run autonomously and when to slow it down. The rest of this guide is the reference material to go deeper.

Part IV: Core concepts, tools, context, and the 200K window

Bob's user interface is conversational, but underneath every response is a structured set of mechanical operations. Bob picks a tool, presents the tool call with its parameters for your approval, runs it, and reads the result back into the conversation. Understanding those mechanics, what tools exist, how Bob manages its context window, what happens when context goes wrong, is the difference between using Bob effectively and fighting it.

The tool taxonomy

Bob exposes six categories of tools. They map roughly to the actions a developer takes during a normal workday: reading files, writing files, running commands, calling external services, switching contexts, and asking for clarification. Each category contains specific named tools, and each tool has well-defined parameters that Bob presents for your approval before running.

Category	Tools	What they do
Read	`read_file`, `search_files`, `list_files`, `list_code_definition_names`	View file contents, regex-search across files, list directory structures, extract class/function/method names. Non-destructive, these read but never modify.
Write	`write_to_file`, `apply_diff`, `insert_content`	Create new files, apply targeted diffs to specific parts of a file, insert content at a specific position. These are the tools that modify your codebase.
Command	`execute_command`	Run shell commands in your workspace. The single most dangerous tool, and the one with the most layered safety controls.
MCP	`use_mcp_tool`, `access_mcp_resource`	Call tools or resources exposed by Model Context Protocol servers, databases, APIs, internal systems.
Mode	`switch_mode`	Change Bob's operating mode (Code, Plan, Ask, Advanced, Orchestrator). Used by Orchestrator mode to delegate subtasks to specialized modes.
Question	`ask_followup_question`	Request clarification when Bob needs more information before proceeding. The escape hatch from ambiguous prompts.

Every tool follows the same approval workflow. Bob analyzes your request, selects the tool, fills in the parameters, and presents the call to you for review. You see exactly what file Bob wants to read, what command Bob wants to run, what diff Bob wants to apply. You approve, reject, or

ask Bob to modify the call. Bob runs the approved version, captures the result, and feeds it back into the conversation as new context.

This pattern is what makes Bob feel like an agent rather than a chat bot. The conversation is not just words, it is a structured sequence of tool calls and results, with Bob deciding what to do next based on what just happened. When Bob hits an error compiling code, it sees the error output through execute_command's result, decides the fix, calls apply_diff to make the change, and re-runs the command. That feedback loop is the core mechanic.

The 200,000-token context window

Bob processes each interaction with a context window of up to 200,000 tokens. The window includes Bob's system instructions and tool definitions, the full conversation history (every message you've sent and every response Bob has produced), the contents of any files referenced with @ mentions, the output of every tool call Bob has made in this conversation, and the descriptions of every MCP tool you have connected.

That last item is more important than it sounds. Every MCP server you connect adds its tool definitions to the context window. A heavy MCP loadout can consume a significant fraction of your budget before you've sent your first message. The official guidance is to disable MCP servers you're not actively using and to enable only the specific tools you need from servers you do use.

Bob exposes the current token usage in real time at the top-right of the chat panel. When a conversation approaches the limit, Bob automatically condenses it at the 140,000-token mark. The condensation logic preserves the most recent and most relevant context, summarizes or removes older conversation segments, and maintains the system instructions and tool definitions. The conversation continues without interruption from the developer's perspective but with a shorter effective history.

This matters for Bobcoin economics. Every token in the context window is processed on every message you send. A long conversation with extensive file references costs more per turn than a short focused one. The official best practice is to start new chats for new tasks rather than letting a single chat sprawl across multiple unrelated goals, both for cost reasons and because mixing tasks invites the failure mode covered next.

Context poisoning, and why no 'wake-up prompt' fixes it

Context poisoning is one of the most candid topics in Bob's documentation, and it is worth reading carefully because it diverges from how most AI coding tools talk about reliability. Bob's docs acknowledge directly that context can become contaminated in ways that degrade output progressively, and that no clever prompt reliably restores a poisoned conversation.

The symptoms are recognizable. Suggestions become nonsensical, repetitive, or irrelevant to your actual question. Tool calls stop matching what you asked for. Orchestrator-mode chains stall, loop, or fail to complete. A corrective prompt seems to fix things for one or two messages,

then the same behavior returns. Bob struggles to use or recall tools that were working fine earlier in the session.

The causes are several. Bob may hallucinate a detail and then treat its own hallucination as factual context for subsequent reasoning. Outdated or misleading comments in the codebase can lead Bob down wrong paths. Logs or text you paste in may contain hidden control characters. And as conversations grow, useful older context gets pushed out by the condensation logic, leaving poisoned data with relatively greater weight.

> **The IBM documentation is unusually direct about this**
>
> "Once bad data enters the context, it tends to persist. Bob reevaluates this tainted information in subsequent reasoning cycles, similar to a permanent flaw affecting perception until the context is completely reset."
>
> And further: "A frequent question from the community is whether a 'wake-up prompt' can resolve context poisoning. Short answer: No."
>
> The only reliable recovery is to start a new chat session. Be selective about what you paste. Break complex tasks into smaller focused sessions so stale information ages out faster. If a tool returns nonsensical data, delete that message before Bob processes it.

This is a notable thing for a vendor to publish in a developer guide. Most AI coding tools market themselves as endlessly steerable through better prompting. Bob's documentation argues the opposite, that there is a class of failure no prompting fixes, and the right response is to recognize it and restart. For developers used to the steerability promise, internalizing this is the difference between a frustrating experience and a productive one.

Working with large projects

The 200,000-token window is generous but it is not infinite. Large monorepos, codebases with thousands of files, or single files that exceed tens of thousands of lines all require deliberate management. The official guidance breaks down into a handful of habits worth internalizing.

- Be specific about files. Use full paths and function names rather than vague references like 'the main file' or 'that handler we discussed'. Bob has no privileged knowledge of your codebase, it sees only what you mention or what its tools return.
- **Target line ranges, not whole files.** Use `@/src/utils/validation.ts:45-67` to include only the lines you actually need. The cost saving is meaningful and the result is usually better-focused.
- Break large tasks into subtasks. Refactoring a thousand-line file works better as ten conversations about pieces of it than as one conversation about all of it. This also helps with context poisoning, fresh contexts age out failure modes.
- Highlight selectively. The Cmd+L / Ctrl+L keyboard shortcut sends highlighted code directly to the chat with file path and line numbers attached. It's the lowest-friction way to give Bob exactly what it needs and nothing more.

- Configure large-file handling. Bob has a setting that controls how it processes files above a threshold size. For codebases with very large files, adjusting this prevents single mentions from consuming the budget.
- Manage MCP tools. Disable any MCP server you're not actively using. Within a server, disable tools you don't call. Each unused tool definition is wasted context.

Part V: Configuration, making Bob fit your team

Bob ships with sensible defaults, but its real power comes from configuration. Three configuration mechanisms (custom modes, custom rules, and the .bobignore file) let you reshape Bob's behavior to match your team's standards, your project's sensitivity, and your individual workflow. A fourth mechanism, the slash-command system, lets you codify repeatable workflows into one-line commands. This chapter covers all four plus the smaller settings worth knowing.

Keyboard shortcuts

Bob's most-used keyboard shortcuts are worth memorizing on day one. They are configurable through the standard VS Code keyboard shortcuts editor (Cmd+Shift+P → 'Preferences: Open Keyboard Shortcuts'), but the defaults are reasonable.

Action	macOS	Windows / Linux
Open / focus Bob chat	⌘ + L	Ctrl + L
Inline edit at cursor	⌘ + K	Ctrl + K
Send selected code to chat	⌘ + L	Ctrl + L
Toggle literate coding mode	⌘ + M or ⌘ + I	Ctrl + M or Ctrl + I
Cycle through modes	⌘ + .	Ctrl + .
Generate (in literate coding)	⌘ + Enter	Ctrl + Enter
Reject (in literate coding)	⌘ + Shift + Backspace	Ctrl + Shift + Backspace

Beyond the defaults, the documentation calls out the Bob.acceptInput command as worth configuring. It has no default binding but submits text or accepts the primary suggestion when triggered, useful for developers who work keyboard-first and want to avoid reaching for the mouse during approval flows. Bind it to Alt+Enter or Ctrl+Space if you find yourself approving a lot of suggestions in sequence.

.bobignore: controlling what Bob can touch

The .bobignore file is the first security boundary you should set up in any non-trivial workspace. It uses the same syntax as .gitignore and tells Bob which files and directories to exclude from its tools. Place it in your workspace root. Bob actively monitors the file and reloads its rules whenever it changes.

```
# Example .bobignore patterns
.env
.env.*
```

```
secrets/
*.key
*.pem
config/credentials.json
build/
node_modules/
dist/
*.log
!important.log    # negation: keep this even though *.log is ignored
```

Bob's interaction with .bobignore is enforced strictly across read tools (read_file, search_files, list_files, list_code_definition_names) and write tools (write_to_file, apply_diff). The execute_command tool also checks a predefined list of file-reading commands like cat and grep against .bobignore. Ignored files appear in file listings with a 🔒 indicator. Attempts to access them produce an explicit error message in chat: "Access to [file_path] is blocked by the .bobignore file settings."

> **Honest limitation IBM documents**
>
> The tools `insert_content` and `search_and_replace` use an internal change-management component whose final write operation may bypass .bobignore rules. Initial read attempts are blocked but the save action itself does not currently have an explicit check.
>
> .bobignore protects against Bob's tools, not the operating system. It is not a sandbox. For true isolation use BobShell's sandboxing (covered in Part VIII).

Custom modes: building purpose-built personas

Bob's five built-in modes (Code, Ask, Plan, Advanced, and Orchestrator) cover the common cases but not every workflow. Custom modes let you create specialized personas with tightly scoped capabilities. A security-review mode that can only read code and write to the findings panel. A documentation mode that can only edit markdown files. A test-engineering mode that has full code access but operates on a specific test directory. Each is defined as YAML, lives either globally or scoped to a project, and shows up in the same mode-selector dropdown as the built-ins.

The minimum custom mode definition is six fields: a slug used internally, a display name, a role definition that describes Bob's personality and expertise, an optional when-to-use clause that Orchestrator mode uses for delegation decisions, a list of tool groups the mode can use, and optional custom rules. Tool groups are: read, edit (optionally restricted by file regex), browser, command, and mcp.

```
# Example: .bob/custom_modes.yaml
customModes:
  - slug: docs-writer
    name: 📝 Documentation Writer
    roleDefinition: |
      You are a technical writer specializing in clear documentation.
      You write for engineering audiences and favor precision over
```

```
    cleverness. You always include code examples for technical claims.
  whenToUse: Use this mode for writing and editing documentation.
  customInstructions: |
    Focus on clarity and completeness. Use second person ('you')
    throughout. Include a minimum of one concrete example per
    non-trivial concept.
  groups:
    - read
    - - edit
      - fileRegex: \.(md|mdx)$
        description: Markdown files only
    - browser
```

Project modes live in .bob/custom_modes.yaml and can be committed to source control to share with the team. Global modes live in custom_modes.yaml in your user directory and apply across all your projects. When the same slug is defined at both levels, the project mode wins. You can also override Bob's built-in modes, defining a custom mode with the slug 'code' replaces the default Code mode for that project. This is useful when, for example, your project requires Python-only edits and you want to enforce that restriction at the mode level rather than relying on developer discipline.

Custom rules: codifying team standards

Custom rules influence how Bob approaches its work without changing what tools it has. Where modes are about capability, rules are about style and convention. You write rules as plain text in a few canonical locations and Bob loads them automatically into its context for matching modes.

The mechanics are flexible. The simplest setup is a single .bobrules file in your workspace root with general guidance. A .bobrules-code file applies only when Code mode is active.
A .bobrules-{slug} file applies to any custom mode whose slug matches. For more structured rule sets, use directories: .bob/rules/ for general rules, .bob/rules-code/ for Code-mode-only rules, .bob/rules-{slug}/ for any mode. Files in directories load in alphabetical order, so naming them 01-style-guide.md, 02-naming.md, 03-testing.md gives you predictable ordering.

Bob also honors AGENTS.md, the convention pioneered by Cursor and adopted by Aider and other AI-first tools. If you have an AGENTS.md file in your workspace root, Bob loads it by default. This makes Bob compatible with the team-standards files you may already have written for other tools. Disable it through the bob-code.useAgentRules setting if you want full control through Bob's own files instead.

Effective rules are specific and actionable. "Format code nicely" is bad guidance. "Use four spaces for indentation in JavaScript files" is good. The documentation gives a representative list of examples:

- "Always use spaces for indentation, with a width of 4 spaces"
- "Use camelCase for variable names"
- "Write unit tests for all new functions"
- "Explain your reasoning before providing code"

- "When adding features to websites, ensure they are responsive and accessible"

For organization-wide standards, the recommended pattern is to maintain a shared rules repository that team members clone into their global ~/.bob/rules/ directory, with project-specific overrides committed to each project's .bob/rules/ directory under version control. Project rules override global rules when they conflict, so the system handles common cases gracefully.

Telemetry

Bob's telemetry posture is unusually privacy-positive among major AI coding tools. Telemetry collection is opt-in, not opt-out, by default Bob does not collect usage data. If you want to help IBM improve the product, you can enable it under Settings → Application → Telemetry, where a single checkbox controls everything. The categories of events collected include task openings and completions, inline completion acceptance and rejection, DevSecOps scan results, conversation messages, and tool invocations.

For enterprise deployments where data handling matters, the opt-in default is worth knowing about. It means a fresh Bob install in a regulated environment is silent unless explicitly configured otherwise. This is different from tools that ship with telemetry on by default and require admin action to turn it off.

Model Context Protocol: extending Bob's reach

Model Context Protocol is the standard that lets Bob connect to external tools and services through a uniform interface. The official IBM analogy is a USB-C port, any MCP-compatible server can plug into any MCP-compatible client. For Bob, MCP is the mechanism for everything that isn't a local file operation: querying internal databases, calling internal APIs, integrating with ticket systems, talking to design tools.

Bob supports MCP at two configuration levels. Global MCP servers live in mcp_settings.json in your user directory and apply across all workspaces. Project-level servers live in .bob/mcp.json in the workspace root and can be checked into version control to share with the team. When the same server name is defined at both levels, the project configuration wins.

Three transport mechanisms are supported. STDIO transport runs the MCP server locally as a child process of Bob, communicating through stdin/stdout, fastest, simplest, and inherently secure because there's no network exposure. Streamable HTTP transport is the modern remote standard, a single HTTP endpoint supporting both request-response and Server-Sent Events streaming. SSE transport is the legacy remote option, kept for backward compatibility but not recommended for new development.

```
# .bob/mcp.json, STDIO server example
{
  "mcpServers": {
    "local-db-query": {
      "command": "node",
      "args": ["scripts/db-mcp.js"],
      "cwd": "${workspaceRoot}",
      "env": {
```

```
        "DB_HOST": "localhost"
      },
      "alwaysAllow": ["query", "schema"],
      "disabled": false
    }
  }
}
```

Bob can also write MCP servers for you, given the right setting. With "Enable MCP Server Creation" turned on, you can ask Bob "Create an MCP tool that gets the current Bitcoin price" or "Build an MCP server to interact with the GitHub Gist API" and Bob will scaffold a TypeScript project, implement the requested tools, prompt for any credentials needed, add the server configuration, and connect to it automatically. For developers who don't want to write MCP servers by hand, this is a meaningful productivity multiplier.

MCP security cannot be ignored

Never hardcode API keys, tokens, or other sensitive credentials directly in MCP configuration files. Use environment variables or a secure credential store. Add any config containing secrets to .gitignore.

Vet external MCP servers carefully. Local STDIO servers run with the same permissions as Bob, they can read your files, call out to the network, and execute arbitrary code on your machine. Review the source, understand what permissions they request, and prefer servers from trusted publishers.

For confidential projects, prefer local MCP servers over remote ones. Remote servers may log or store your data in ways you do not control.

Part VI: The fifteen features that define daily use

Beyond modes and configuration, Bob ships with a set of features that show up in daily work. The official documentation lists fifteen of them under Features in the IDE nav, and each is worth understanding because each removes a specific kind of friction. This chapter walks through them in order of how often you'll touch them.

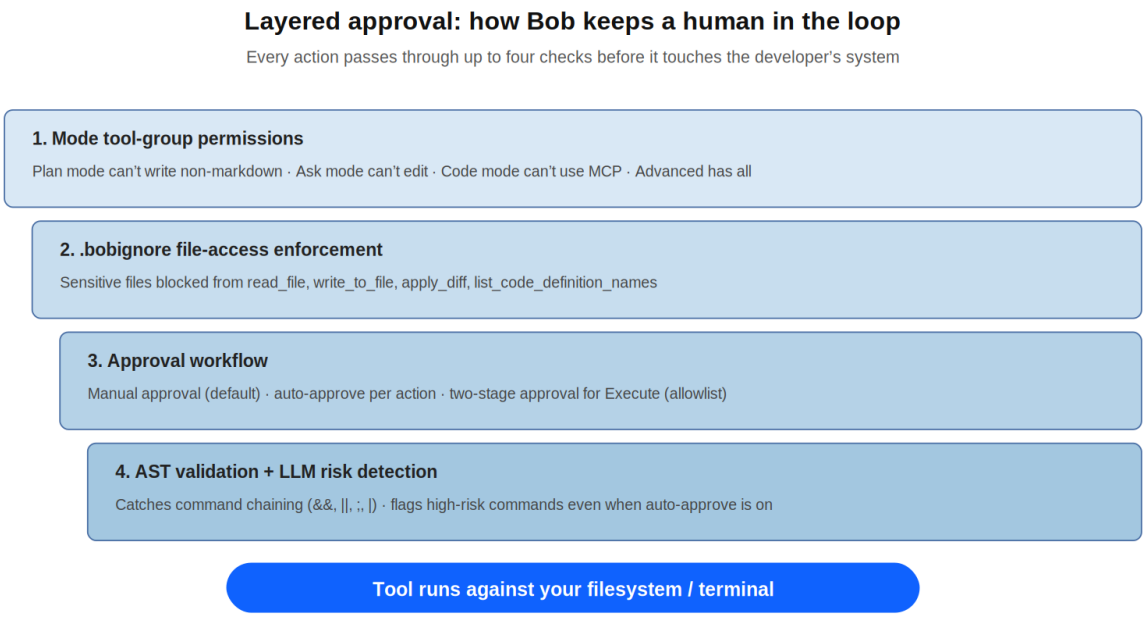
The chat interface

The chat interface is the surface you interact with most. Beyond the basics covered in Part III, modes, mentions, slash commands, enhance prompt, a few behaviors are worth knowing. File paths and URLs in the chat history are clickable: a click opens the file in the editor or the URL in your browser. Code blocks have a dedicated copy button. Long messages can be clicked to collapse and expand, which matters in long sessions where you want to scroll past completed work. Loading spinners, red error messages, and green success messages give visual feedback as tools run.

The most common failure mode is writing prompts that are too vague. The documentation explicitly compares two prompts side by side: "Fix the code" versus "Fix the bug in calculateTotal that returns incorrect results when given negative numbers." The second works dramatically better not because Bob is stupid but because the first leaves too much for Bob to guess. The single highest-leverage habit in chat usage is being specific about what's broken, what should change, and what good looks like.

Auto-approve: speed versus safety

The auto-approve toolbar lets you grant Bob permission to take specific kinds of actions without explicit per-action approval. It speeds up workflow significantly, when Bob is reading thirty files to understand a codebase, manually approving each read is tedious. It also significantly increases risk, particularly for the Write and Execute actions.



There are nine actions you can auto-approve, each with a different risk profile.

Action	Risk	What it does
Read	Medium	Bob can read files and list directories without asking. Useful for codebase exploration.
Write	High	Bob can create, modify, and delete files without asking. The single most consequential toggle.
Browser	High	Bob can navigate websites, click elements, submit forms. Can perform unintended actions on real services.
MCP	Medium-High	Bob can call MCP tools without asking. Risk depends on what each MCP server can do.
Mode	Low	Bob can switch modes automatically based on task. Affects workflow, not system access.
Subtasks	Low	Bob can create and complete subtasks for complex work. Affects orchestration, not system access.
Execute	High	Bob can run terminal commands without asking. Powerful and dangerous in equal measure.
Question	Medium	After a timeout, Bob picks the first answer to an ask_followup_question. Useful for unattended runs.

Action	Risk	What it does
Todo	Low	Bob can update its internal todo list without asking. Task management only.

Execute auto-approval is the most consequential and has the most safety layers. It uses a two-stage approval model: first you enable Execute auto-approval at the toolbar level, then individual commands must be added to an allowed list before they run. When Bob tries a new command, it still prompts, you can approve once or add a pattern to your allowed list. This prevents auto-approve from being a blank check while still saving time on routine commands.

Two additional safeguards sit underneath Execute auto-approval. Bob includes LLM-based risk detection that analyzes each command for potential security concerns before running it, high-risk commands may require additional confirmation even with auto-approve on. And Bob uses AST (Abstract Syntax Tree) parsing on shell commands to detect chaining via &&, ||, ;, or | operators, which catches command injection attempts and ensures pattern matching against your allowed list works on the actual structure of the command rather than just its text.

The recommended discipline is to enable Read auto-approval once you're comfortable, keep Write and Execute on manual unless you have a specific reason to relax them, and never enable auto-approval in production-adjacent environments. The official guidance is blunt: "Never auto-approve in production."

Modes in detail

Part I introduced the five built-in modes. The reference detail matters when you're deciding which mode to use for a specific task. Each mode has a published role definition, a specific tool-group access pattern, and a primary use case.

Mode	Tool access	Role and use
💻 Code	read · edit · command	Default mode. Highly skilled software engineer persona. For implementing features, fixing bugs, refactoring.
❓ Ask	read · browser · mcp	Knowledgeable technical assistant. Cannot modify files. For explanations, analysis, exploration.
📝 Plan	read · edit (markdown only) · browser · mcp	Inquisitive technical leader. Can write planning documents but not code. For architecture, design, breakdown.
🛠️ Advanced	all tool groups	Same persona as Code but with unrestricted tool access including browser and MCP. For complex workflows requiring everything.

Mode	Tool access	Role and use
🔀 Orchestrator	none (delegates)	Strategic workflow orchestrator. Cannot run tools directly, coordinates by delegating to other modes. For multi-step projects spanning specialties.

Switching modes is fast: dropdown selector, slash command (/code, /ask, /plan, /advanced, /orchestrator), keyboard shortcut (Cmd+. on Mac, Ctrl+. elsewhere, each press cycles through modes), or by accepting mode-switch suggestions that Bob offers when it thinks a different mode would handle a request better.

The reason mode constraints matter is the principle of least privilege. Plan mode can't accidentally edit a non-markdown file because the tool group doesn't permit it. Ask mode can't accidentally run a destructive command because it has no command access. These aren't suggestions Bob tries to follow, they are hard constraints at the tool layer. You can rely on them as part of your safety model.

Context mentions: the seven types

Context mentions are the single most important feature for getting good output. They let you point Bob at specific elements of your project rather than describing them in prose. Bob supports seven types of mentions.

Mention	Format	What it includes
`@/path/to/file.ts`	File path from workspace root	Full file contents with line numbers. Add :start-end for a line range.
`@/path/to/folder`	Folder path from workspace root	Contents of all files directly in the folder. Non-recursive.
`@problems`	Keyword	All errors and warnings from Bob's Problems panel, grouped by file.
`@terminal`	Keyword	The last command and its complete output from the active terminal.
`@a1b2c3d`	Git commit hash	Commit message, author, date, and complete diff.
`@git-changes`	Keyword	git status output and the diff of all uncommitted changes.
`@https://...`	Full URL	Fetched and converted to markdown for context.

The dropdown that appears when you type @ filters recent files, visible folders, recent git commits, and the special keywords. The dropdown also filters out node_modules, .git, dist, and out automatically, though you can still type those paths manually if you need them.

One subtle behavior to know: file and folder mentions bypass .bobignore. The reasoning is that an explicit mention from the developer is a strong intent signal that should override the file-access policy. Git-related mentions like @git-changes respect .gitignore because they rely on git commands that already honor it. If you want a file truly off-limits, .bobignore protects against tool usage but not against explicit @ mentions; for hard exclusion you would also need to remove the file from the workspace or use OS-level permissions.

Bob tips: real-time code-quality analysis

Bob tips runs continuously in the background, analyzing every open file for code-quality issues. It calculates two metrics per function: cyclomatic complexity (the number of independent paths through the code, where 10 or higher is considered high) and maintainability index (how easy the code is to read, test, and extend, where 70 or lower is considered low).

Functions that exceed quality thresholds are marked with a purple underline in the editor. Hovering shows a tooltip with the issue and AI-powered refactoring suggestions. The findings also flow into a centralized Bob Findings panel where you can navigate across files. Each finding has a "Fix with Bob" button that opens a chat conversation pre-loaded with the relevant context, so refactoring becomes a one-click affair.

Bob tips runs without explicit invocation. There is no scan command. You write code, and Bob continuously evaluates it. This is the closest Bob comes to being a passive presence rather than an active agent, it's working in the background regardless of whether you're talking to it.

Slash commands: workflow shortcuts

Slash commands give you single-keystroke access to workflows. Three are built in. /init initializes a workspace with Bob, setting up the .bob directory, creating starter configuration files, and connecting Bob to the project. /review runs a code review on your changes; it has several variants: /review on its own reviews uncommitted changes, /review <branch> compares against a branch, /review <issue-url> --issue-coverage validates that your changes align with a GitHub issue. /create-pr generates a pull request with an AI-written description based on your branch's changes.

You can create custom commands by adding markdown files to .bob/commands/ in your workspace or ~/.bob/commands/ globally. The filename becomes the command name, security-review.md becomes /security-review. The file body is the prompt Bob will use when invoked. YAML frontmatter at the top lets you add a description (shown in the command menu) and an argument-hint (shown as light gray placeholder text).

```
---
description: Create a new API endpoint with error handling
argument-hint: <endpoint-name> <http-method>
---
```

```
Create a new API endpoint called $1 that handles $2 requests.
Include proper error handling, input validation, and JSDoc
comments. Follow the project's existing endpoint patterns
in /src/api/.
```

Custom commands shine when paired with custom modes and custom rules. A team can codify its standard PR-creation workflow as /our-pr, its security-review workflow as /sec-review, its release-prep checklist as /release-prep, each one a markdown file under version control that any team member can invoke. This is one of Bob's strongest features for team standardization.

Checkpoints: undo for AI

Checkpoints solve the most uncomfortable problem with agentic AI: what happens when the agent does something you didn't want, multiple steps deep into a session, and now you want to go back. Bob's answer is a shadow Git repository, separate from your project's actual Git history, that snapshots your workspace before every file modification. You can view diffs against any checkpoint and restore your files to a previous state, either keeping the conversation context or rolling that back too.

The architecture is genuinely clever. Bob initializes a hidden Git repository in a sidecar location, not in your project's .git. Every time Bob is about to modify a file, the shadow repo captures the current state. Restoration uses git's underlying reset and checkout machinery. The shadow repo respects .gitignore (so node_modules and build artifacts aren't snapshotted) and ships with built-in exclusions for media files, .env, and other low-value content. Git LFS patterns from your .gitattributes are honored. Nested git repositories disable checkpoints automatically with a clear notification.

There are two restore options. "Restore files" reverts only the workspace files without modifying the conversation history, useful when you want to compare alternative implementations or roll back code while keeping the chat context. "Restore files & task" reverts both code and conversation back to the checkpoint, which is destructive of the chat history beyond that point and requires confirmation.

> **When checkpoints are the right answer**
>
> If Bob starts producing subpar work mid-session, the documentation explicitly recommends restoring to a previous checkpoint over trying to correct course with new prompts. Restoring resets context contamination. New prompts don't.
>
> Checkpoints only capture changes made during active Bob tasks. Manual edits outside of Bob actions are not snapshotted, so don't rely on checkpoints for changes you make yourself.
>
> Checkpoints require Git to be installed on your system. They use the simple-git library under the hood.

Code actions: Bob in the gutter

Code actions are Bob's hooks into the lightbulb icon that VS Code shows in the editor gutter when quick fixes are available. Select code, click the lightbulb (or use Cmd+./Ctrl+.), and you get five Bob-specific actions: Add to Context sends the selection to the active chat with file path and line numbers; Explain Code asks Bob to explain the selection; Improve Code asks Bob to suggest improvements; Inline Chat opens a chat interface directly at the cursor position for quick exchanges; Move to Chat sends the selection to the main chat panel.

The same actions are accessible through the right-click context menu (Bob submenu) and through the Command Palette (Cmd+Shift+P, then type 'Bob'). The Command Palette version applies to the active chat task if one exists, which makes it useful for continuing an in-flight conversation rather than starting fresh.

Next Edit: experimental completion

Next Edit is Bob's competitor to the Copilot-style inline-completion experience. It analyzes your code context and suggests the next change as an inline diff with gutter indicators and jump decorations. It also detects diagnostic errors near your cursor and offers automatic fixes. It is currently marked experimental in the IDE, accessible through the Completion Mode dropdown in either Quick Settings or the full settings panel.

If you're coming from Copilot or Cursor, Next Edit will feel familiar. The experience differs in being diff-oriented rather than single-line, Bob proposes changes as patches you accept or reject rather than ghost text that you accept-as-you-type.

Code reviews: pre-commit, pre-PR

Bob's code review feature gives you AI-assisted review before you commit or open a PR. Two modes: Branch Comparison compares your changes against any branch (default or remote), and Issue Coverage validates that your local changes match the requirements of a specific GitHub issue. Both produce findings in the Bob Findings panel (bugs, security concerns, performance issues, style inconsistencies) each with severity, location, and a Fix-with-Bob button.

The dedicated Review Panel in the sidebar provides a visual interface, but you can also trigger reviews entirely through slash commands: /review for uncommitted changes, /review <branch> for branch comparison, /review <issue-url> --issue-coverage for issue validation. The chat-driven approach is faster for keyboard-first developers.

Findings have three statuses (Open, In Progress, Resolved) and four actions (Fix with Bob, Mark as Resolved, Mark as Open, Dismiss). You can configure review exclusions through glob patterns, useful for excluding test files, generated code, or configuration files that don't need review. The recommended workflow is to use Bob's review as a first pass before human code review, treating it as additive rather than substitutive.

Commit messages and pull requests

Two adjacent features automate the routine writing around Git operations. Commit message generation lives as a sparkle icon next to the commit message input in the Source Control panel, stage your changes, click the icon, and Bob writes a message based on the diff. It analyzes your branch name and recent commit history to match your project's conventions, which means it picks up patterns like conventional-commits prefixes automatically if your repo uses them. If you don't like the first suggestion, click again for alternatives.

Pull request generation is similar but bigger. You can trigger it through the PR icon in the Source Control panel, through the Command Palette ("Bob: Generate PR"), or through the /create-pr slash command. Bob asks which base branch to target, suggests the three most likely candidates, then generates a PR title and description based on your branch's commits and diff. Bob searches six standard locations for a PR template in your project and uses the matching one, pull_request_template.md, docs/pull_request_template.md, .github/pull_request_template.md, and three more. If multiple templates exist, Bob asks which to use. If none exist, Bob uses a built-in default with title-from-branch-name, change summary, implementation details, testing info, and relevant links.

Both features benefit from atomic commits. The more focused each commit is, the better the generated messages and PR descriptions. Sprawling commits produce vague summaries.

Enhance prompt: refining vague requests

Enhance prompt is the sparkles icon at the bottom right of the chat input. Type a vague request, click the icon, and Bob rewrites your prompt with clarity, structure, and explicit requirements. The documented example is the difference between "make a function to calculate tax" and a structured five-bullet specification that lists parameters, return value, error handling, and a request for usage examples.

The feature is marked experimental and the documentation is honest that the enhanced version may not always match your intent. The recommended workflow is to enhance, review the enhanced version, edit if needed, then send. You can also apply enhancement multiple times to iteratively refine. The feature works best when your initial prompt has a clear goal but lacks structure, it cannot create intent from nothing.

Literate coding: instructions in the source file

Literate coding lets you write instructions in plain language directly inside your source file, exactly where the code should go. Toggle the mode with Cmd+M (Mac) or Ctrl+M (Windows/Linux), or click the magic wand icon in the editor toolbar. Your instructions appear highlighted in blue to distinguish them from regular code. Click Generate (Cmd+Enter), and Bob replaces your instructions with real code, showing an inline diff for review.

The workflow shines for in-context edits where the natural place to express intent is inside the file. "Add a function below that validates the email format from the User type defined above" works well as a literate coding comment because Bob can see both directions of context

naturally. Acceptance is Cmd+Enter, rejection is Cmd+Shift+Backspace. Code is written into the file before acceptance so you can test it before committing.

The current limitation is single-file scope. Literate coding works on one file at a time; multi-file support is on the roadmap. For changes that span files, the chat panel is still the right surface.

Skills: codifying reusable workflows

Skills are reusable instruction sets that teach Bob new workflows. Think of them as recipes that Bob follows consistently for specific types of work, code reviews, documentation generation, security audits, feature implementations following a team standard. A skill lives as a directory under .bob/skills/ (project) or ~/.bob/skills/ (global), with a SKILL.md file at the root and optional supporting files.

The SKILL.md format is straightforward: YAML frontmatter with required name and description fields, then the skill's instructions in markdown. The description is critical because Bob uses it to decide when to activate the skill, skills without clear descriptions are ignored. Supporting files in the skill directory (checklists, templates, severity guides, examples) are loaded automatically when the skill activates, so you can keep SKILL.md focused on the core workflow while putting reference material in companion files.

```
---
name: security-review
description: Review code for security vulnerabilities and best practices
---

Perform a security-focused code review.

1. Check for common vulnerabilities:
   - SQL injection risks
   - XSS vulnerabilities
   - Authentication / authorization issues
   - Sensitive data exposure

2. Review security best practices:
   - Input validation
   - Output encoding
   - Secure configuration
   - Error handling

3. Provide findings with:
   - Severity (Critical, High, Medium, Low)
   - Location in code
   - Recommended fix
   - Reference to standards

Use the severity guide in `severity-levels.md` for classification.
```

Skills are only available in Advanced mode, because skills often need access to multiple tool groups to run their workflows effectively. By default Bob asks permission before activating a

skill, "Always allow skills" is a settings toggle if you want skills to run automatically when their descriptions match the task at hand.

The SKILL.md format is the same convention used by Anthropic's Claude skills system, so authoring skills for Bob produces artifacts that may be portable to other Claude-based tools. This is consistent with IBM's pattern of building on open conventions where they exist (MCP, AGENTS.md, gitignore syntax) rather than inventing proprietary ones.

Bobalytics: usage visibility

Bobalytics is the dashboard that gives developers and teams visibility into how Bob is being used. It tracks Bobcoin consumption over time, shows which features are driving usage, surfaces productivity patterns. For individual developers on Pro/Pro+/Ultra plans, Bobalytics is the way to understand whether you're trending toward your monthly allocation. For enterprise admins, it's how you track spending across users and teams and identify high-cost patterns worth optimizing.

Bobalytics is referenced from the product marketing page as the cost-visibility companion to Bobcoin metering. The product side is more developed than the documentation side at the moment, usage analytics being one of the areas where the docs are clearly still maturing.

Part VII: The complete Bob commands reference

Bob exposes its functionality through five distinct command surfaces: slash commands in the chat interface, mode-switching commands, keyboard shortcuts, BobShell CLI flags, and configurable settings keys. Each surface answers a different question, what can I ask Bob to do in chat, how do I tell Bob to behave a certain way, what shortcut wakes Bob up, how do I script Bob from the terminal, and how do I configure Bob to my taste. This chapter is the consolidated reference.

Three notes before the tables. First, all commands and shortcuts come from the public IBM documentation at bob.ibm.com/docs and the IBM Bob GitHub repository. Where commands changed between Project Bob and GA, the GA naming is used. Second, slash commands behave consistently across Bob-IDE and BobShell: the same /init you type in the IDE chat works in the terminal session. Third, custom commands extend this list. You can add any command you want by dropping a markdown file into the right directory, which is covered in the second half of this chapter.

Slash commands: chat-driven actions

Slash commands are the fastest way to invoke common workflows. Type / in any chat input (Bob-IDE chat panel or BobShell prompt) and Bob shows a fuzzy-searchable menu of available commands. Three are built in and ship with every Bob install.

Command	What it does	Common variants
`/init`	Scans the current project and generates an AGENTS.md file at the workspace root. The file becomes persistent project context that Bob loads on every future conversation. Setting this up once is one of the highest-leverage things you can do per project.	Single form. No arguments.
`/review`	Runs an AI-driven code review. Detects bugs, security issues, performance problems, and style inconsistencies. Produces findings in the Bob Findings panel with severity, location, and a Fix-with-Bob button on each.	`/review` (uncommitted), `/review <branch>`, `/review #123 --issue-coverage`, `/review <issue-url> --issue-coverage`
`/create-pr`	Generates a pull request title and description from your branch's commits and diff. Asks which base branch to target. Honors a PR template if found in any of six standard repository locations (.github/pull_request_template.md and similar).	Single form. No arguments.

Beyond the three built-ins, mode-switching slash commands let you change Bob's operating mode from inline rather than reaching for the mode dropdown.

Mode command	Switches to	When to reach for it
`/code`	Code mode	Default development mode. Implementing features, fixing bugs, refactoring.
`/ask`	Ask mode	Read-only exploration. Asking about the codebase without risk of modification.
`/plan`	Plan mode	Architecture and breakdown. Writes markdown plans but not code.
`/advanced`	Advanced mode	All tools unrestricted. Complex multi-tool workflows.
`/orchestrator`	Orchestrator mode	Multi-mode coordination. Delegates subtasks to specialized modes.

Custom slash commands

Any markdown file you drop in .bob/commands/ (project-scoped) or ~/.bob/commands/ (global) becomes a slash command. The filename becomes the command name. review.md becomes /review (overriding the built-in if it conflicts). security-audit.md becomes /security-audit. The file body is the prompt Bob will execute when the command is invoked.

Three things make custom commands powerful. First, you can include frontmatter that gives the command a description (shown in the autocomplete menu) and an argument-hint (placeholder text in the input). Second, you can reference positional arguments with $1, $2, $3, the user types /security-audit api server and your prompt sees api as $1 and server as $2. Third, the prompt body can include @ context mentions, which means you can encode workflows like "review @src/api/ and validate against @docs/api-standards.md" as a single command. Custom commands live under version control with your project, which makes them the single best mechanism for team-wide workflow standardization.

```
---
description: Security audit for API endpoints with severity scoring
argument-hint: <component-name>
---

Perform a security audit of $1.

Steps:
1. Read all files under @/src/$1/
2. Check against @/docs/security-checklist.md
3. Generate findings with severity (Critical, High, Medium, Low)
4. For each finding, propose a fix with code
```

```
Use @/docs/owasp-references.md as the reference standard.
```

Once that file lives at .bob/commands/security-audit.md, every team member with the project checked out can type /security-audit api and get the same workflow against the same standards. The skill side of Bob (covered in Part VI) handles longer reusable workflows; custom commands handle quick, parameterized actions you reach for daily.

Keyboard shortcuts: every default binding

Bob uses VS Code's keyboard shortcut system. The defaults below are documented; all are remappable through Cmd+Shift+P → 'Preferences: Open Keyboard Shortcuts'. Conflicts with imported VS Code or Cursor bindings are flagged at first launch.

Action	macOS	Windows / Linux
Open or focus Bob chat panel	⌘ + L	Ctrl + L
Send selected code to chat (with file path + line numbers)	⌘ + L	Ctrl + L
Inline edit at cursor	⌘ + K	Ctrl + K
Toggle literate coding mode	⌘ + M or ⌘ + I	Ctrl + M or Ctrl + I
Cycle through Bob modes	⌘ + .	Ctrl + .
Generate (inside literate coding)	⌘ + Enter	Ctrl + Enter
Reject (inside literate coding)	⌘ + Shift + Backspace	Ctrl + Shift + Backspace
New chat task	⌘ + N	Ctrl + N
Open Command Palette (then type 'Bob')	⌘ + Shift + P	Ctrl + Shift + P
Open code-action lightbulb on selection	⌘ + .	Ctrl + .

Two additional commands worth binding manually because they have no defaults: Bob.acceptInput (sends or accepts the active suggestion, useful for keyboard-driven approval flows; bind to Alt+Enter or Ctrl+Space) and Bob.newTask (explicit new conversation while preserving the current; useful for parallel investigations).

BobShell CLI flags

BobShell is the terminal surface. Beyond the conversational bob command, a handful of flags shape every invocation. The reference below captures the documented flags from bob.ibm.com/docs/shell.

Flag	What it does	Example
`-p "prompt"`	Non-interactive mode. Sends a one-shot prompt and returns the result. Read-only by default, Bob cannot modify files unless --yolo is also passed.	`bob -p "explain this codebase"`
`--yolo`	Enables write access in non-interactive mode. Limited to the directory where Bob was started, a hard guarantee at the tool layer, not a soft preference.	`bob -p "fix the bug" --yolo`
`-s, --sandbox`	Enables sandboxing. Combines with environment variables to pick Seatbelt profile or container backend.	`bob -s "analyze this script"`
`--auth-method`	Selects authentication method. The api-key option is the standard for CI/CD.	`--auth-method api-key`
`--model`	Override the routing layer's model selection. Mostly useful for testing or for tasks where a known model fits best.	`--model granite-code`
`--version`	Print version and exit.	`bob --version`
`--help`	Print usage and exit. The fast way to verify install.	`bob --help`

BobShell also responds to environment variables for persistent configuration. BOB_API_KEY supplies the credential for non-interactive auth. BOB_SHELL_SANDBOX takes a boolean or a specific backend name (docker, podman, sandbox-exec) to set sandboxing without a flag. SEATBELT_PROFILE picks among the five Seatbelt profiles. SANDBOX_FLAGS passes through to the container runtime for resource limits (memory, CPU). These are the right surfaces for embedding Bob in scripts and pipelines without command-line clutter.

Settings keys worth knowing

Bob's settings live in Bob-IDE's Settings panel or in JSON form at the workspace or user level. Most defaults are fine; a handful are worth changing intentionally.

Setting	What it controls	Default
`bob.telemetry.enabled`	Whether usage telemetry is collected. Opt-in.	false

Setting	What it controls	Default
`bob.autoApprove.read`	Auto-approval for file reads. Generally safe to turn on.	false
`bob.autoApprove.write`	Auto-approval for file writes. Keep manual unless you have a reason.	false
`bob.autoApprove.execute`	Auto-approval for terminal commands. Use a tight allowlist if enabled.	false
`bob-code.useAgentRules`	Honor AGENTS.md convention used by other AI tools. Disable for Bob-only rule management.	true
`bob.completion.mode`	Inline completion mode. Set to Next Edit for experimental diff-based completion.	off
`bob.findings.exclusions`	Glob patterns excluded from code reviews. Add test fixtures, generated code.	[]
`bob.skills.alwaysAllow`	Skip per-skill approval prompts. Convenience over security.	false
`bob.mcp.serverCreation`	Allow Bob to scaffold new MCP servers on request.	false
`bob.context.largeFileThreshold`	Size above which a file gets special-cased handling. Useful for monorepos.	varies

The single most important configuration discipline is the auto-approval set. Keep read manual in your first week, then move to read auto-on, write manual, execute manual. Move execute auto-on only with a tight allowlist of safe commands (git status, npm test, ls). Never put rm, git push --force, or anything that touches production credentials on an allowlist.

Part VIII: Designing custom Bob modes

Part VI introduced the five built-in modes and gave a quick example of a custom one. This chapter is the deep dive: every field in a mode definition, how tool groups and file regex work together, how modes interact with rules and skills, and three complete worked examples drawn from real workflows: a documentation architect mode (modeled on IBM's published tutorial), a security reviewer, and an IBM i developer persona for RPG modernization.

The reason custom modes deserve their own chapter is that they are the single most powerful customization point in Bob. A custom mode is a persona, a personality, a permission set, and a behavior contract all in one declaration. Once you have a few well-designed modes for your team, Bob stops feeling like a general-purpose assistant and starts feeling like a set of specialized colleagues who each know their job. That shift is where the productivity numbers in the IBM case studies actually come from.

Anatomy of a mode definition

Custom modes live in YAML. Project modes go in .bob/custom_modes.yaml in the workspace root. Global modes go in custom_modes.yaml in your user directory. When the same slug is defined at both levels, the project definition wins. You can override the built-in modes (slug: code, ask, plan, advanced, orchestrator) by defining custom modes with those slugs, a useful technique when a project needs stricter defaults than the platform ships.

A mode definition has seven fields. Three are required (slug, name, roleDefinition), four are optional (whenToUse, customInstructions, groups, source).

Field	Required	What it does
`slug`	Yes	Internal identifier. Lowercase, no spaces. Used in slash commands (/your-slug) and in mode override (a custom mode with slug: code replaces the built-in Code mode).
`name`	Yes	Display name in the mode-selector dropdown. Free text; emoji is conventional. Example: '📝 Documentation Writer'.
`roleDefinition`	Yes	Bob's personality and expertise for this mode. Free-form prose. Treat it like a job description, who is this Bob, what does this Bob care about, what standards does this Bob hold.
`whenToUse`	No	Hint to Orchestrator mode about when to delegate to this mode. Omit for modes you only invoke explicitly.
`customInstructions`	No	Behavior rules. More tactical than roleDefinition, specific dos and don'ts. Loaded into context on every interaction in this mode.

Field	Required	What it does
`groups`	No	Tool groups this mode can access. Constraining groups is how you build read-only modes, markdown-only modes, etc. Omit for full access.
`source`	No	Metadata about where the mode came from. Useful for shared mode libraries; can be omitted in most cases.

Tool groups and file regex

The groups field is the security and capability boundary. Five tool groups exist: read, edit, browser, command, mcp. Listing a group gives the mode access to every tool in that group. Omitting the field gives the mode all groups (equivalent to Advanced mode's permissions).

The edit group is special because it accepts a file regex constraint. Where a plain edit entry permits Bob to write any file, a constrained edit entry only permits writes that match a regex pattern. This is how Plan mode is built (edit constrained to markdown) and how you build mode personas that should be expressive in writing but cannot touch source code.

```
# Tool group syntax in YAML
groups:
  - read                          # Plain entry: full access
  - browser
  - mcp
  - - edit                        # Tuple entry: edit + constraint
    - fileRegex: \.(md|mdx|rst)$
      description: Documentation files only
  - - command                     # Same tuple pattern for command
    - regex: ^(git|npm|yarn|pnpm) # If supported in your Bob version
      description: Git and package managers only
```

The exact tuple-form constraints supported by command group vary by Bob version; check the documentation for your install. The edit regex is universally supported and is the most useful constraint in practice. A common pattern is to write a docs-only mode with edit constrained to *.md, a tests-only mode with edit constrained to **/test/**, a config-only mode with edit constrained to *.yaml or *.json. Each of these encodes a real workflow contract: the developer can grant write access without worrying that Bob will edit something it shouldn't.

Worked example one: documentation architect mode

This example is modeled directly on IBM's published "AI Code Documentation with IBM Bob" tutorial, which walks through creating a docs architect mode that analyzes a GitHub repository and produces structured documentation artifacts. The mode pattern shown here is the production-quality version: tightly scoped, well-defined responsibility, file-regex protection against accidental code edits.

```
# .bob/custom_modes.yaml
customModes:
  - slug: docs-architect
    name: 📚 Documentation Architect
    roleDefinition: |
      You are an experienced technical writer and documentation
      architect. You analyze codebases to identify documentation
      gaps, produce structured artifacts (README, API docs,
      architecture diagrams in mermaid, runbooks), and reorganize
      sprawling docs into navigable hierarchies.

      You write for engineers as the primary audience. Your tone is
      precise, complete, and free of marketing language. You always
      include a concrete code example for any non-trivial concept.

    whenToUse: |
      Use this mode when the user needs to document an existing
      codebase, fill documentation gaps, restructure existing docs,
      or generate API references.

    customInstructions: |
      - Always produce a documentation plan before writing files.
        The plan goes in docs/_plan.md.
      - Mirror the source repository structure in the docs tree.
      - Prefer many small focused files over single sprawling ones.
      - When unclear, mark sections [NEEDS-INPUT] rather than
        guessing, never hallucinate API signatures or behavior.
      - Mermaid diagrams for any flow with more than three steps.

    groups:
      - read
      - browser
      - mcp
      - - edit
        - fileRegex: ^docs/.*\.(md|mdx)$|^README\.md$
          description: Documentation files only
```

Three observations about this mode worth lifting out. The roleDefinition gives Bob a persona with a documented bias, engineering audience, no marketing language, examples required. The customInstructions are tactical commitments, produce a plan first, mirror source structure, mark uncertainty rather than guessing. The edit regex prevents this mode from accidentally touching anything outside the docs tree or the root README, which is exactly the protection you want when running documentation work on a codebase you also actively develop.

The IBM tutorial also makes a broader point worth absorbing: the mode is most useful when paired with a folder of supporting templates and a style guide. Put .bob/skills/docs-architect/README-template.md and similar starter files in your project, and the mode picks them up automatically. The mode is the persona; the supporting files are the toolkit.

Worked example two: security reviewer mode

```
customModes:
  - slug: security-reviewer
```

```
    name: 🔒 Security Reviewer
    roleDefinition: |
      You are a senior security engineer specializing in application
      security and secure code review. You review code for OWASP
      Top 10 vulnerabilities, injection risks, authentication and
      authorization flaws, sensitive data exposure, and supply chain
      issues. You hold engineers to a high bar without lecturing.

    whenToUse: |
      Use this mode for pre-commit security review, audit of recently
      modified code, or proactive review of untrusted dependencies.

    customInstructions: |
      - Always produce findings as a table with: severity
        (Critical/High/Medium/Low), location (file:line), summary,
        recommended fix.
      - Use OWASP IDs where applicable (e.g., A03:2021 - Injection).
      - For each Critical or High finding, propose the fix as a
        diff that can be applied directly.
      - Mark findings DISMISSED only if you can articulate a clear
        compensating control elsewhere in the codebase.

    groups:
      - read
      - browser
      - - edit
        - fileRegex: ^docs/security/.*\.md$
          description: Only writes to security documentation
```

This mode demonstrates a different pattern. Edit access is severely restricted, security-reviewer can read the entire codebase but cannot modify source code at all. It can only write to a docs/security/ directory where findings reports go. Engineers use this mode to get a structured security pass over their work, then implement the fixes themselves in Code mode. The separation is deliberate: a mode that finds problems should not also be the mode that fixes them, because the temptation to skip review is too high.

Worked example three: IBM i developer mode

This example anticipates the iMode capability planned for IBM Bob Premium Package for i (covered in Part XI). Until iMode ships natively, IBM i shops can build a meaningful approximation as a custom mode that captures platform-specific knowledge in the roleDefinition and customInstructions, with a TechChannel article reporting that the IBM i Premium Package will ship with about 40 such curated skills.

```
customModes:
  - slug: ibm-i-developer
    name: 🟢 IBM i Developer
    roleDefinition: |
      You are an experienced IBM i (formerly AS/400, iSeries)
      developer with deep knowledge of RPG (fixed-form and free-form),
      CL, SQL/DB2 for i, COBOL, and DDS. You understand library
      lists, source physical files, the IFS, journaling, commitment
      control, and the platform's call stack semantics.
```

```
    You favor free-form RPG over fixed-form when modernizing,
    embedded SQL over native I/O for new development, and
    idiomatic IBM i patterns over forced abstraction from other
    platforms.

  whenToUse: |
    Use this mode for RPG modernization, IBM i SQL development,
    CL scripting, fixed-to-free conversion, and documenting
    legacy IBM i applications.

  customInstructions: |
    - Never invent RPG op-codes. If unsure, say so explicitly.
    - For fixed-to-free conversion, preserve the original logic
      exactly; flag GOTO usage but do not silently restructure
      control flow without permission.
    - Reference IBM i SQL using fully qualified names with the
      appropriate schema (library) prefix.
    - For new RPG, prefer subprocedures over global variables,
      and prefer service programs over copy-and-paste reuse.
    - Generate matching DSPF (display files) when a program
      references screen logic that has no DSPF in the project.

  groups:
    - read
    - browser
    - mcp
    - - edit
      - fileRegex: \.(rpgle|sqlrpgle|clle|dds|dspf|prtf|pgmsrc)$
        description: IBM i source file extensions only
```

Two notes on this mode that may matter for IBM i practitioners. First, the file regex captures the common extensions used when IBM i source is checked into Git (rpgle, sqlrpgle, clle, dds, dspf, prtf). For shops that work directly in source physical files on the IBM i itself, this constraint is mostly aspirational, the iMode capability is still planned, with TechChannel reporting the IBM i Premium Package will provide native source-member awareness when it ships. Second, the customInstructions deliberately encode community-known best practices, embedded SQL over native I/O, subprocedures over globals, service programs over copy-paste, that newer RPG developers don't always pick up by default. A team's IBM i mode is also its house style guide.

Distributing modes across a team

Custom modes are most valuable when they're shared. Three distribution patterns work in practice.

The first pattern is committing project modes to source control. .bob/custom_modes.yaml lives in your workspace; check it in alongside your code. Every team member who clones the repository picks up the modes automatically. New hires immediately have the same documentation architect, security reviewer, and platform-specific personas the rest of the team uses. This is the lowest-friction model and works well for projects with a stable team and a single set of conventions.

The second pattern is a shared mode library distributed through a separate repository. A team maintains a modes-library repo with a curated set of modes for the organization. Developers clone the repo into their global ~/.bob/ directory or symlink specific modes into their projects. This works well when the same modes apply across many projects and when changes to the modes need to roll out independently of any one project's release cycle.

The third pattern, supported by IBM in the enterprise tier, is an admin-managed mode set distributed through Bob's enterprise configuration. The admin defines modes in the central admin dashboard; team members receive them automatically. This is the right model for regulated environments where mode definitions need to be audited and version-controlled at the organization level.

In all three patterns, the mode set is part of the project's or organization's intellectual property. Treat it that way, version it, review changes, document why each mode exists. A team's mode library is, over time, a more meaningful encoding of how the team works than any prose documentation could be.

Part IX: Learning Bob beyond the docs, the YouTube channel and community

IBM maintains a YouTube channel dedicated to Bob, and a growing community of independent creators publishes tutorial content on their own channels. Reading the documentation gets you the structured reference. Watching demos gets you the texture of how Bob actually feels to use, what the modes look like in practice, how the chat panel responds, what a real refactoring session looks like end-to-end. Both matter. This chapter is the orientation map for the video material.

The official IBM Bob YouTube channel

IBM publishes regularly to a dedicated IBM Bob YouTube channel. The cadence is roughly weekly during the GA launch period, slowing to bi-weekly afterward. The content mix is intentional: short feature demos (three to five minutes, focused on a single capability), longer end-to-end walkthroughs (ten to twenty minutes, modernization or new-build scenarios), and customer case-study videos (variable length, real production work). Below are the categories you'll find on the channel as of mid-2026, with representative titles drawn from the public archive.

Category	What's covered	Representative titles
Feature demos	Single-capability walkthroughs showing one Bob feature end-to-end. Best starting point if you're new to Bob.	Meet Bob: An AI-Powered IDE for Modern Software Development; IBM Bob Tutorial + Demo: How to Use Modes in the IDE; What is IBM Bob? (Ep-01)
IBM i / RPG	Mainframe and midrange-platform work. Includes fixed-to-free conversion, RPG documentation generation, and AGENTS.md templates for IBM i.	IBM Bob - IBM i Generate Documentation for RPG; IBM BOB - Using AI to document old RPG Source Code
Mainframe / Z	COBOL, PL/I, JCL, Assembler refactoring and analysis. Drives the Premium Package for Z value proposition.	Beyond the Demo: Testing IBM Bob AI with COBOL, PL/I, Assembler, and JCL (third-party Planet Mainframe)
Customer stories	Real client work, Blue Pearl, Novacomp, APIS IT, EY. Production deliverables rather than toy examples.	AI-led modernization, delivered with IBM Bob: The Blue Pearl story; Novacomp client modernization with IBM Bob

Category	What's covered	Representative titles
Tutorials / how-tos	Step-by-step builds. Document architect mode, RAG implementation, agent skills authoring.	AI Code Documentation with IBM Bob; My First Hands-On Experience: From Planning to RAG Implementation

If you have an hour to invest in video learning, the highest-value path is: "What is IBM Bob? (Ep-01)" for orientation, then the Modes tutorial to see all five built-in modes in action, then a customer case study that matches your industry. For IBM i developers, the RPG documentation video is the most direct demonstration of what Bob does for that platform. For mainframe developers, Planet Mainframe's independent assessment is honest about both strengths and limitations and is worth the full read or watch.

Independent creators worth following

Several independent creators and bloggers have built useful Bob-focused content. Three are worth singling out by name.

Nick Litten, an IBM i veteran and prolific blogger, published "IBM BOB – The King of IBM-i Code Modernization?" in April 2026, a hands-on test of Bob against authentic 1990s RPG code, including GOTO tags and monolithic subroutines. The piece walks through fixed-to-free conversion with real before-and-after examples. For RPG developers specifically, it's one of the most concrete demonstrations available.

Niklas Heidloff, who works on generative AI at IBM, maintains a blog at heidloff.net with frequent Bob coverage. His December 2025 post "Boost Developer Productivity with IBM Bob" frames Bob in the context of the broader vibe-coding debate, useful for thinking about where AI development tools genuinely add value versus where the hype exceeds reality. Heidloff is also one of the more reliable sources for tying Bob's product decisions to the IBM AI engineering team's broader strategy.

Thomas Suedbroecker publishes hands-on walkthroughs at suedbroecker.net, including "My First Hands-On Experience with IBM Bob: From Planning to RAG Implementation", a substantive demonstration of building a Retrieval-Augmented Generation system with Bob's help, covering the planning phase, code generation, and documentation flow. The post is one of the cleaner end-to-end examples available outside IBM's own marketing.

Greg Hintermeister's piece "IBM Bob, HOLY BUCKETS This is Amazing!" on Medium is enthusiastic but honest about Bob's failure modes. He recounts a session where Bob removed a non-working word cloud feature from the running app but left the underlying code untouched, a pattern that, repeated across a long session, could produce a wasteland of dead code. This kind of independent observation is exactly what the IBM marketing material won't include, and reading it before committing to a long Bob session is worth the time.

Community channels and forums

Beyond YouTube and personal blogs, three community spaces matter for ongoing Bob conversation. The IBM Bob community forum at community.ibm.com hosts the most active practitioner discussion, including AGENTS.md template sharing, mode definitions, and bug reports. The Planet Mainframe site at planetmainframe.com publishes the most rigorous independent testing of Bob against legacy mainframe codebases. Stack Overflow has begun accumulating Bob-tagged questions, though the volume is still modest compared to GitHub Copilot.

For people who learn best through structured curriculum, the IBM Think 2026 keynote sessions on YouTube include "Granite 4.1, IBM Bob & the quantum computing future" (Episode 105 of the Mixture of Experts series) which puts Bob in the broader context of IBM's AI strategy. It's the right starting point if you want to understand where Bob fits in IBM's portfolio before diving into the product itself.

> **A note on the video evidence**
>
> Video demonstrations have a known bias: they show the cleanest path through a feature, not the path most developers actually take. A modernization demo that works in nine minutes on YouTube may not work that smoothly on your codebase, with your context, with your team's standards.
>
> Watch videos to understand what Bob is doing, not to set expectations for how long things will take in your environment. The case studies, Blue Pearl, Novacomp, APIS IT, are more honest about that gap because they describe real production deliverables with real timelines, including the upfront context-setting work that the demos skip.

Part X: Use cases, where Bob earns its keep

Productivity-percentage marketing claims (45 percent average, 70 percent on certain teams, 90 percent on specific tasks) are easy to dismiss because they're abstract. The point of this chapter is to make them concrete: what does Bob actually do in production environments, against what kinds of problems, with what kinds of inputs and outputs. The material that follows draws on public customer stories from IBM, independent reviews, and the documented use cases that recur across the Bob ecosystem.

Twelve use cases organized into four categories: modernization, new development, governance and security, and platform-specific work. For each, a brief setup, the concrete scenario, what Bob does, and where the limits are.

Modernization use cases

Use case 1: Java version upgrade across a large codebase

The canonical Bob proof point, drawn from IBM's published Blue Pearl case study and the company's coverage in IBM's GA launch materials. Blue Pearl, a South African IT consultancy with a flagship Java application called Blue App matching 26,000 consultants to enterprise clients, used Bob to upgrade from Java 11 to Java 21. The team spent roughly three hours up front constructing a detailed prompt with architecture diagrams, AWS workflow information, endpoint definitions, inter-service dependencies, and data flow maps. From there, Bob produced a dependency map across the Java codebase, surfaced deprecated API calls, highlighted outdated libraries, and exposed integration points that had never been formally documented. The full upgrade, work that would have taken a thirty-day engineering effort, completed in three engineering days, with 127 deprecated API calls resolved, 92 percent test coverage established, and zero post-deployment defects, per the published case study.

The honest framing matters: Blue Pearl's three-hour context-setting work was the unsung enabler. Bob is not a thirty-second prompt-and-go tool for modernization at this scale. The productivity gain comes from amplifying disciplined upfront context with rapid execution, not from skipping the planning.

Use case 2: Replatforming a layered monolith to microservices

From IBM's published Novacomp case study. Novacomp, an IT consultancy, took on a structural upgrade for an internal change-data-capture (CDC) operations system, replatforming a layered logical monolith of enterprise REST APIs into a cloud-native microservices architecture. The applications consisted of REST APIs written in older Java with legacy dependency trees that had become costly to maintain and difficult to secure. The work would typically take a three-to-five-person Novacomp team a couple of months. With Bob, per the case study, one engineer completed the structural upgrade in two days. Bob analyzed dependency trees, identified the natural service boundaries based on data flow, generated the new service scaffolding, migrated the routing layer, and preserved the business logic contracts.

Where Bob earns its keep here is in the discovery phase. A human engineer reading the codebase to find service boundaries would spend most of the time on it; Bob produces a candidate decomposition in minutes that the engineer reviews and refines. The execution phase, generating boilerplate, moving code, wiring up routing, is fast for both human and Bob, but the discovery acceleration is the differentiator.

Use case 3: Documenting legacy code with no living author

APIS IT, the Croatian government IT agency, used Bob to document mission-critical systems spanning decades, a twenty-year-old EGL/CIS system, decades-old JCL and PL/I batch jobs, and a sprawling network of internal services. The reported outcome: documentation produced in hours rather than weeks, with 100 percent operator-verified accuracy in Croatian-language descriptions of system behavior. For organizations where the original developers have retired and the documentation never existed, Bob's ability to read the code and produce structured explanations is more valuable than the modernization itself, because it unlocks the modernization work that comes next.

The pattern is generalizable. Any organization with code older than ten years, where institutional knowledge has decayed, can use Bob to produce living documentation as a precursor to either modernization or simple ongoing maintenance. The Planet Mainframe assessment of Bob against COBOL, PL/I, and Assembler confirms this is one of Bob's most reliable strengths: even where Bob's refactoring suggestions are imperfect, its explanation of what existing code does is consistently strong.

Use case 4: Fixed-form to free-form RPG conversion

For IBM i shops, the most common modernization use case is RPG conversion, taking 1990s-era fixed-form RPG (often RPGIII) and converting to free-form RPGLE or SQL-embedded SQLRPGLE. Independent IBM Champion Nick Litten demonstrated this against a genuine 1990s-vintage program with GOTO tags and monolithic subroutines, and Bob produced a clean free-form version that preserved the original logic exactly. The honest caveat from the Planet Mainframe COBOL/PL/I/Assembler testing applies here too: Bob occasionally defaults to older control-flow constructs that are technically correct but don't reflect modern best practices, and explicit guidance through custom rules or a custom mode (Part VIII covers this) substantially improves the output.

The expected workflow is: convert the program with Bob, review the output, run regression tests against the legacy version, iterate. For developers who have been doing fixed-to-free conversion by hand for years, Bob accelerates the boring middle while leaving the verification work where it belongs, with the developer.

New development use cases

Use case 5: Greenfield API service from a written specification

A developer with a clean specification for a new REST API can build the scaffolding, routing, validation, error handling, and tests in a single Bob session. The typical pattern is Plan mode

first, Bob reads the specification, produces a structured plan including data models, endpoint contracts, and test strategy, then Code mode for implementation, then Ask mode for explanation of design decisions to capture in documentation. For an experienced developer, this collapses what would be a two-day setup into a half-day. For a non-traditional developer (covered in the citizen-developer chapter), it may be the only path to shipping a working service.

Use case 6: Building a Retrieval-Augmented Generation system

Thomas Suedbroecker's blog post "My First Hands-On Experience with IBM Bob" walks through building a RAG system with Bob's help. The scenario is typical: the developer knows what a RAG system needs (a vector store, document chunking, an embedding pipeline, a retrieval-then-generation flow) but not the specific library APIs or the optimal chunking strategy. Bob produces the scaffolding, suggests the libraries, generates the chunking logic, and produces working integration code. The developer reviews and refines. The end state is a working RAG prototype in hours rather than days.

This use case generalizes to any well-understood-architecture but unfamiliar-stack situation. Bob bridges the gap between developers who know the concepts and the specific libraries that implement them. Mistakes still happen, chunking strategies that produce unhelpful retrievals, embedding choices that don't match the use case, but the iteration cost is low enough that the bridge is faster than learning the libraries cold.

Use case 7: Generating tests for existing code

One of the consistently strong use cases across customer stories: feed Bob a function or class and ask for a complete test suite. Bob produces unit tests, edge-case tests, and where appropriate integration tests, organized by the testing framework you indicate. For codebases with poor test coverage, Blue Pearl reported moving from low coverage to 92 percent during their Bob-driven modernization, this is one of the highest-leverage uses of Bob's time.

The honest limitation: Bob writes tests that pass against the existing implementation. If the implementation is buggy, the tests will codify the bug. Best practice is to combine test generation with the security-reviewer or code-review mode (Parts VI and VIII) to surface logic issues before writing tests, then generate tests against the corrected implementation.

Governance, security, and compliance use cases

Use case 8: Pre-commit security review

The /review and /review --issue-coverage commands (Part VII reference) give developers an AI-driven first pass over their changes before opening a PR. The Northdoor Bobathon team specifically highlighted security awareness as one of Bob's strongest features, security being secure by default at the point the code is written, rather than added as a separate review step after the fact. For organizations where security review has been a bottleneck in the development cycle, shifting the first pass to AI before human review materially shortens cycle time.

The right framing is that Bob's security review is additive, not substitutive. A human reviewer is still required for any non-trivial change, especially anything touching authentication, payments,

or sensitive data. Bob catches the things humans miss when tired; humans catch the things Bob doesn't yet understand.

Use case 9: Compliance documentation generation

Regulated industries (financial services, healthcare, government) require documentation that demonstrates compliance with specific standards (PCI-DSS, HIPAA, FedRAMP). Bob can generate that documentation by reading the code and mapping behaviors to the relevant compliance controls. The published Novacomp work and APIS IT government work both touched this. APIS IT's mission-critical government infrastructure required exactly this kind of documentation, and Bob produced operator-verified accurate output.

The recurring pattern in compliance use cases is that the documentation is treated as the audit artifact. Bob produces it; a human reviewer signs off; the artifact lives alongside the code under version control. For organizations facing audit cycles, this transforms documentation from a manual task into a generated artifact reviewed for accuracy.

Use case 10: Audit-traceable agent action logs

BobShell records every action the agent takes as a self-documenting, traceable process. For regulated industries where auditability is the gating requirement for AI in production code, this is a structural advantage that less-governed competitors don't match. The use case here is less a single workflow and more an organizational capability: deploying Bob in a context where every prompt, every tool call, every file modification can be reconstructed for post-incident forensic analysis or routine compliance review.

Platform-specific use cases

Use case 11: Mainframe COBOL modernization with the Z Premium Package

The IBM Bob Premium Package for Z, currently in technical preview, brings Architect mode for system-level reasoning across mainframe applications and a Z-aware Code mode for COBOL, PL/I, and JCL work. The independent Planet Mainframe assessment by Uwe Graf put this through its paces. PL/I support was consistently strong, even for complex programs shaped by decades of style variations. Assembler support stood out, Bob frequently suggested cleaner, more modern alternatives to historically motivated but hard-to-read constructs. COBOL refactoring was good with a caveat: Bob occasionally defaults to older control-flow constructs like GO TO that are technically correct but don't align with modern COBOL best practices.

The takeaway for Z teams: engage with the technical preview now. The capability is real, the limitations are documented, and the iteration loop with IBM is open while the product is still in preview. By the time the Z Premium Package is GA, the organizations that participated in preview will have substantial production experience that competitors will lack.

Use case 12: IBM i Developer Mode and the upcoming Premium Package for i

TechChannel reported in late May 2026 that IBM is shipping the Premium Package for IBM i on June 24, 2026, with five specialized capabilities including an IBM i Developer Mode persona

configured for RPG, CL, SQL/DB2 for i, COBOL, and DDS. The package includes approximately 40 curated skills addressing IBM i-specific use cases: documentation generation, fixed-to-free conversion, modernization workflows, and platform-specific best practices.

For IBM i shops, the right move is two-track. Track one: use Bob today with the IBM i developer custom mode shown in Part VIII for current RPG and CL work, knowing the experience is approximate rather than native. Track two: prepare for the Premium Package for i release in late June 2026 by engaging with IBM about the upgrade path, identifying the use cases in your shop that map to the 40 curated skills, and being ready to switch when the package ships. The Premium Package is what would shift Bob from a useful general tool to a category-defining IBM i development partner.

Where Bob does not earn its keep

Honest discussion of use cases requires the inverse: where is Bob not the right answer. Three patterns recur in independent assessments.

First, Bob is not the cheapest answer for autocomplete-style assistance. Developers who want fast inline completions for the kind of work that's already 80 percent muscle memory may find Copilot's flat-rate subscription cheaper than Bob's Bobcoin-metered model. The economics of Bob favor multi-step agentic work; for line-by-line autocomplete, the value differential shrinks.

Second, Bob is not yet a replacement for native IBM i development tooling. Until the Premium Package for i ships and matures, IBM i shops working in source physical files rather than Git-managed sources should keep their existing tooling alongside any Bob evaluation. The Code for IBM i extension on top of VS Code with GitHub Copilot remains a mature option that Bob's general-purpose capabilities do not yet fully replicate.

Third, Bob is not a substitute for human judgment on architectural decisions. The Greg Hintermeister anecdote, Bob removing functionality from the running app but leaving the underlying code intact, illustrates a pattern that repeats in long agentic sessions. Bob is excellent at completing well-specified tasks. Bob is less excellent at recognizing when a task specification is incomplete, when the right answer is to stop and ask a deeper question. That judgment remains with the developer, and the developers who get the most out of Bob are the ones who maintain that judgment rather than outsource it.

The combined picture is that Bob is most valuable for teams doing substantive, multi-step development work in heterogeneous environments where governance matters, exactly the audience IBM has been targeting since the GA launch. It is less valuable, though still useful, for solo developers doing simple work in well-supported stacks, and it is not yet valuable for IBM i source-physical-file work until iMode lands.

Part XI: BobShell, agentic AI in the terminal

BobShell is the second major surface in the Bob ecosystem, and for some developers it's the primary one. If you live in the terminal, Vim users, CLI-first devops engineers, people doing infrastructure work, BobShell may be more important than Bob-IDE. It brings the same agent, same tools, same modes, same configuration to the command line, with two additional capabilities that the IDE doesn't have: native scriptability and built-in sandboxing.

Installation

BobShell installs independently of Bob-IDE. The official documentation supports three installation paths: an installation script, your platform's package manager, or, if you have Bob-IDE installed, the editor's command palette. On Linux and macOS, the install script is the simplest path. On Windows, the package manager approach is recommended.

After installation, verify with bob --version. First-run requires login with your IBMid (same account as Bob-IDE) and acceptance of the license agreement. For automation environments where interactive login isn't practical, BobShell supports API-key authentication through the --auth-method api-key flag, which is the path most CI/CD pipelines will use.

Interactive vs non-interactive

BobShell runs in two modes that feel quite different. Interactive mode (bob, with no arguments) opens a conversational REPL in your terminal, you type, Bob responds, you approve tool calls one by one. It's the closest the terminal experience comes to the IDE chat panel. Non-interactive mode (bob -p "your prompt") executes a single prompt and returns the result, optimized for scripting and CI/CD.

Interactive session

```
# Start an interactive session
bob

# Type your question or request
Explain the functionality in @src/main.js

# Bob explores, asks permission to read files,
# and responds. Use slash commands like /help, /code,
# /ask just like in the IDE.
```

Interactive mode supports the same @ context mentions, the same slash commands, the same modes. Tool approvals are presented in the terminal, you approve, decline, or modify each one as it comes up. Diffs for file changes are rendered as colored text in the terminal by default; the /editor command lets you configure an external editor for reviewing diffs in a more comfortable surface.

Non-interactive session

```
# One-shot prompt
bob -p "Explain this project"
```

```
# Pipe stdin
cat buildError.txt | bob -p "Explain this build error"

# Redirect stdout
bob -p "Review @bigFile.java" > review.md

# Reference files in the project
bob -p "Summarize the functionality in @src/main.js"
```

The --yolo flag

By default, non-interactive mode runs in read-only mode, Bob can analyze but cannot modify files. The `--yolo` flag enables write access in non-interactive mode.

Important constraint: even with --yolo enabled, BobShell will not write or update files outside the directory where it was started. This is a hard guarantee at the tool layer, not a soft preference Bob might violate.

```
bob -p "Fix bugs in @app.js" --yolo
```

For CI/CD use, the typical pattern is to wrap BobShell in a script that sets BOB_API_KEY, runs the relevant bob -p commands, and processes the output. Output formatting hints in your prompts help downstream processing: "Enclose the answer in markdown tags" or "Format the response as JSON" produces parseable output.

Sandboxing: the differentiator

BobShell ships with built-in sandboxing, a feature that is genuinely differentiated from competitors and may be the single best reason to prefer BobShell over alternatives for security-conscious work. Two sandboxing mechanisms are supported. macOS Seatbelt is built into the OS and used by Apple's own sandbox-exec utility, with five distinct profiles you can choose between. Container-based sandboxing uses Docker or Podman and works cross-platform with complete process isolation.

Seatbelt profile	Network	Write access	When to use
permissive-open (default)	Allowed	Project only	Everyday development work.
permissive-closed	Blocked	Project only	Offline work, sensitive environments where outbound network is not desired.
permissive-proxied	Via proxy	Project only	Corporate environments where traffic must route through a proxy.

Seatbelt profile	Network	Write access	When to use
restrictive-open	Allowed	Strict limits	High-security work with potentially malicious code that still needs network.
restrictive-closed	Blocked	Maximum restrictions	Maximum isolation. Untrusted code requiring complete network and filesystem separation.

Enabling sandboxing is straightforward and works at three levels with clear precedence.

```
# Method 1: command flag (one-time)
bob -s "analyze this shell script for potential security issues"

# Method 2: environment variable (session-wide)
export BOB_SHELL_SANDBOX=true
export BOB_SHELL_SANDBOX=docker    # or podman, or sandbox-exec

# Method 3: settings.json (persistent, project-wide)
{
  "tools": {
    "sandbox": true
  }
}

# Seatbelt profile selection
export SEATBELT_PROFILE=restrictive-open

# Container customization
export SANDBOX_FLAGS="--memory=4g --cpus=2"
```

Precedence runs highest-to-lowest: command flag, environment variable, settings file. This is the standard pattern for tools that need both persistent defaults and per-run overrides.

The honest limitations are documented. Sandboxing reduces but does not eliminate risk, always review Bob's actions before approval. GUI applications typically won't work inside container sandboxes due to display isolation. Symbolic links pointing outside the project directory may not work. Some Seatbelt profiles block all network. Localhost services on your host may not be accessible from container sandboxes without additional configuration. VPN-connected resources may not be available.

None of this should put you off using sandboxing, it should inform when to use which profile. For everyday work the default permissive-open is fine. For working with code from a stranger on the internet, restrictive-closed inside a Docker container is the right move. The flexibility is the value.

Other BobShell features worth knowing

BobShell shares most features with the IDE but has a few of its own.

- **Checkpointing** is supported in BobShell with the same shadow-Git architecture as the IDE. You can view diffs and restore previous states from the CLI.
- **Slash commands** include the IDE built-ins plus shell-specific ones. /help, /editor, and the mode-switching commands are the most commonly used.
- **Memory files** let you persist context across sessions. Useful for long-running projects where you don't want to re-establish context every time you start a session.
- **The /instance command** manages BobShell instances when you have multiple running concurrently.
- **Trusted folders** control which projects can use BobShell's full capabilities, a separate security boundary on top of sandboxing.
- **IDE integration** lets BobShell run inside the terminal panel of supported editors with shared configuration. The most natural setup is BobShell running inside Bob-IDE's integrated terminal.

For developers whose default surface is the terminal, BobShell may be the entire reason to adopt Bob. The combination of agentic capability with sandboxing depth is hard to match in other tools.

Part XII: Security, sandboxing, and the honest disclosures

This chapter pulls together the security-relevant material that appears across the rest of the guide and adds the parts not covered elsewhere, including the disclosed vulnerabilities and the broader threat model. The framing is deliberate. Bob's security story is genuinely good in places and incomplete in others, and a developer guide that treats both honestly is more useful than one that does either alone.

The security layers Bob actually provides

Bob's security model is built in layers, each addressing a different category of risk.

Layer	What it protects against	Where it's documented
Mode tool-group constraints	Bob using a tool the current mode shouldn't have access to	Built-in modes table in Part VI
.bobignore	Bob reading or writing files marked sensitive	Part V configuration chapter
Approval workflow (manual / auto / two-stage)	Bob taking destructive action without developer consent	Part VI auto-approve section
AST command validation	Command-injection attacks through chained operators	Part VI auto-approve section
LLM risk detection on commands	High-risk commands slipping through auto-approve allowlists	Part VI auto-approve section
BobShell sandboxing	Bob's tools affecting anything outside a controlled environment	Part VII BobShell chapter
MCP security guidance	Untrusted external integrations exfiltrating data or executing arbitrary code	Part V configuration chapter
Opt-in telemetry	Usage data being collected without consent	Part V configuration chapter
IBMid authentication	Unauthorized access to Bob from compromised endpoints	Part III install chapter

Taken together, this is a more thorough security posture than most AI coding tools offer at the same price point. The combination of mode constraints, ignore-file enforcement, layered approvals with AST validation, and BobShell sandboxing covers most of the realistic threat surface for an AI agent operating inside a developer's environment.

The honest disclosures

Bob's history is not unblemished. In January 2026, three months before general availability, security researchers disclosed that Bob could be manipulated into executing malware via the CLI, and that the IDE was vulnerable to common AI-specific data-exfiltration vectors. This was reported in The Register's launch coverage and is part of the public record of Bob's security history.

The right way to read this is not as a reason to avoid Bob but as context for how to use it carefully. The specific vulnerabilities disclosed in January are presumably addressed in the GA release, vendors do not ship products with publicly disclosed unpatched flaws, but the broader category of risk remains. AI agents that can execute commands and read files are inherently risky tools. The safety features Bob ships with are the response to that risk, not an elimination of it.

The documentation is also explicit about other limitations. .bobignore protects against tool access but does not create a system-level sandbox. The insert_content and search_and_replace tools may bypass .bobignore on write operations due to current limitations in their save mechanism. MCP servers run with Bob's permissions and require vetting. Auto-approve in production environments is explicitly discouraged. These are limitations, documented as limitations, rather than capabilities marketed as guarantees.

A working security posture

For a developer using Bob on a real project, the recommended security setup is straightforward and worth establishing on day one.

9. Create a .bobignore file in every workspace. Include .env, .env.*, secrets/, *.key, *.pem, and anything else sensitive. Add to source control so the rules are shared with the team.
10. Keep auto-approval off for Write and Execute. Enable Read auto-approval once you're comfortable, but stay manual for the high-risk actions until you have a specific reason to relax.
11. Use BobShell with sandboxing for any code you don't fully trust. The five Seatbelt profiles give you a graduated response, pick the strictest one that lets the work happen.
12. Vet MCP servers before connecting them. Read the source if open source. Understand what they access. Prefer local STDIO servers over remote ones for confidential projects.
13. Never paste secrets into prompts. Store them in environment variables or secret managers. Reference them by name, not value.
14. Review Bob's actions before approval. The most basic security practice is the most reliable one.
15. Keep Bob updated. Subscribe to release notes via the changelog and update promptly when security fixes ship.

This is the same posture you would adopt for any agentic AI tool, Claude Code, Cursor's agent, Codex CLI all require similar discipline. Bob's advantage is that more of the safety controls are built in by default rather than left to the developer to construct.

Part XIII: Pricing, Bobcoins, and the enterprise plan

Bob's pricing model is unusual enough to require its own chapter. It is a credit system rather than a flat subscription, the credits map roughly to compute but not directly to model tokens, and the enterprise tier introduces a shared-pool model that changes the math significantly. Understanding the economics is part of evaluating whether Bob fits your team.

The Bobcoin

Bob is metered in Bobcoins. One Bobcoin is worth $0.50 USD. Each action you take with Bob (generating code, analyzing files, running commands) consumes some number of Bobcoins based on the computational resources used, which depends on which model the routing layer selected, how many tokens the input and output contained, and what tools were invoked.

The Bobcoin is deliberately an abstraction. Different models consume tokens at different rates, but Bob standardizes the billing into one unit you can budget against. You do not need to think about tokens. You think about Bobcoins, and Bob handles the conversion. Your remaining balance is visible in real time at the top-right of the chat panel.

Plan	Monthly cost	Bobcoins included	Additional support	Best for
Free trial	$0 (30 days)	40	—	Evaluation
Pro	$20 / user / month	40	+ $3 / month	Solo developers, light usage
Pro+	$60 / user / month	160	+ $9 / month	Most teams, moderate usage
Ultra	$200 / user / month	500	+ $30 / month	Heavy modernization, intensive usage
Enterprise	Contact sales	Shared pool	Priority	Organizations, centralized control

Overage works on pay-as-you-go pricing. If you exceed your monthly allocation, additional usage is billed at the per-Bobcoin rate. The base allocation resets monthly. This is the area where the economics get interesting: a Pro user who consistently runs over should consider Pro+, where the per-Bobcoin price drops. An Ultra user pulling 500 Bobcoins consistently is at roughly $0.40 per Bobcoin equivalent, Bobcoins get cheaper at scale, which is the standard volume-discount pattern.

How fast Bobcoins burn

Bobcoin consumption per task is not officially published as a fixed table, but a few patterns emerge from how the routing layer works. Simple completions on a small file with the Granite SLM family consume relatively few Bobcoins. Complex multi-step orchestration with Claude in Advanced mode consumes more. A large MCP-heavy context window with many tool definitions adds overhead to every message even before the substantive work begins. Long conversations consume progressively more per turn because each turn re-processes the accumulated context.

The practical implication for budgeting is that lightweight uses (autocomplete-style assistance, small file edits, single-mode Q&A) extend a Bobcoin allocation a long way. Heavy uses (large modernization tasks, multi-file refactors with extensive context, Orchestrator-mode workflows coordinating multiple modes) consume Bobcoins quickly. A Pro user who tries to do a full Java 16-to-22 modernization the way Blue Pearl did will likely run out long before completing it; that workload belongs on Ultra or Enterprise.

Bobcoins versus tokens

The Bobcoin abstraction matters because it changes the cost-monitoring conversation. With tools that bill in tokens directly (or that pass through API costs from underlying providers), developers and managers need to understand token economics, input vs output token pricing, model-specific multipliers, context-window growth effects. With Bobcoins, you watch one number.

The trade-off is some loss of visibility. You don't see exactly which model handled which task or what the underlying token cost was. Bobalytics provides aggregate consumption patterns but does not expose per-call model selection. For most teams this trade-off is favorable, predictable budgeting beats granular accounting. For teams that want to optimize aggressively at the token level, the abstraction is a limitation.

The Register's launch coverage flagged a real economic risk worth understanding. GitHub recently reworked Copilot pricing because complex multi-step prompts were costing more in inference than the subscription was bringing in. Bob's entire value proposition is multi-step agentic work, which means it starts from a steeper baseline of inference cost per active user. The Bobcoin overage model is IBM's solution: rather than absorb the cost differential and risk margin compression, IBM pushes cost back to the user as work gets harder. This is fair pricing, but it means heavy users should expect their bills to reflect their usage.

The enterprise plan

The Enterprise tier is qualitatively different from Pro/Pro+/Ultra. It introduces shared resource pooling and centralized administration suitable for organizations rather than individuals. The pricing is custom (contact sales) but the structural features are documented.

- **Shared Bobcoin pool.** Instead of per-user allocations, the organization buys a pool of Bobcoins and administrators distribute them to team members. Reallocation can happen in real time, moving Bobcoins from a quiet team to a busy one is a few clicks.

- **Admin dashboard.** Centralized control over spending, user management, team assignment, individual spending limits, and usage insights. Track entitlements and balances across the organization.
- **Team organization.** Group users into teams with role assignments. Modify teams and push instant updates without requiring users to take action.
- **Single sign-on.** Enterprise SSO integration through your identity provider, replacing per-user IBMid sign-in for centralized identity management.
- **Priority support.** Dedicated assistance from IBM Bob specialists, rather than community-supported support.
- **Flexible scaling.** Purchase additional Bobcoin packs as the team grows. The pool model means usage spikes don't trigger per-user upgrades.

For organizations evaluating Bob seriously, the Enterprise tier is almost certainly the right structure, both for cost reasons (the pool model absorbs uneven usage) and for governance (centralized billing, SSO, admin controls). The Pro/Pro+/Ultra tiers are designed for individual developers or very small teams. The Enterprise tier is what you adopt when Bob becomes part of the organization's tooling.

Part XIV: Bob for citizen developers and low-code practitioners

A word of context first. Bob's public documentation does not describe a dedicated no-code or low-code user mode. It is positioned as a developer tool and assumes a developer persona, someone working in an IDE, editing files, running commands, reviewing code. There is no Bob-branded equivalent of Power Platform, OutSystems, or Mendix sitting alongside the developer surfaces. This chapter is therefore interpretive guidance rather than documented capability: where Bob's developer-focused design is approachable to citizen developers, where it is not, and how a non-traditional developer might use Bob productively given what Bob actually offers.

What 'citizen developer' means in the Bob context

The term covers a spectrum. At one end is the business analyst who has never written code but has clear domain problems they want to solve, a finance professional who needs to automate a spreadsheet workflow, an operations lead who wants a small internal tool, a marketing manager who wants to wire two SaaS products together. At the other end is the technically literate non-developer, someone who can read code, has used Excel formulas or SQL, has perhaps configured an iPaaS workflow, but does not write code as a profession.

Bob is not a tool for the first group. There is no visual builder, no drag-and-drop interface, no spreadsheet-style formula language. A finance professional with no technical background would find Bob's IDE-driven workflow unfamiliar at best and intimidating at worst. The right tool for that persona is a true no-code platform, Microsoft Power Apps, Glide, Bubble, or a category-specific tool like Airtable or Zapier.

Bob is, however, increasingly approachable for the second group. The technically literate non-developer, someone who can describe what they want and read the result, can do meaningful work with Bob through three features in particular: literate coding, the natural-language chat, and skills. The rest of this section is for that audience.

The features that lower the bar

Three Bob features bring the cost of entry down significantly for non-traditional developers.

Natural-language chat is the surface most associated with no-code style interaction. You describe what you want; Bob writes it. The same techniques covered in Part VI apply: be specific, use context mentions to point Bob at the right files, break complex tasks into smaller ones. For non-traditional developers, the most useful pattern is to use Ask mode (read-only) heavily before switching to Code mode. Ask mode lets you explore the codebase, understand what's there, ask Bob what would happen if you changed something, without any risk of breaking the project. Once you understand the situation, you switch to Code mode for the actual change.

Skills (covered in Part VI) are the most underrated tool for low-code use of Bob. A skill is a markdown file that encodes a workflow. A non-traditional developer who has identified a repeatable task, say, generating a standard report from a CSV every Monday morning, can describe that workflow once in a skill file, with help from a more technical colleague if needed, and invoke the skill in future runs without re-explaining. The skill carries the technical detail; the user provides the trigger.

A realistic low-code workflow with Bob

To make this concrete, here's what a productive Bob workflow looks like for a non-traditional developer who wants to build a small internal tool. The example is intentionally modest: a Python script that pulls data from a Google Sheet, performs basic analysis, and emails a summary to a recipient list. A developer would write this in an hour. A citizen developer might struggle for a week. With Bob, the citizen developer can plausibly ship it in a half-day.

16. Install Bob-IDE. Skip the import-from-existing-editor step. Sign in with IBMid.
17. Start in Plan mode. Describe the goal: "I want a Python script that connects to Google Sheets, reads a specific sheet by URL, computes the average and median of one column, and emails the result to a list of people every Monday morning. I have admin access to a Gmail account and can create API credentials. Plan how to build this."
18. Bob produces a step-by-step plan. The plan will identify the libraries needed (gspread, google-auth, smtplib), describe the credential setup, lay out the script structure, and suggest a scheduling mechanism. Read the plan. Ask Bob to clarify anything confusing.
19. Switch to Code mode. Ask Bob to create the project structure and write the script following the plan. Approve each file creation. Bob will write the code, including comments explaining what each section does.
20. Read the code. You don't need to deeply understand Python, but read enough to follow what each step does. If something looks wrong, ask Bob about it. Use the explain-code action on sections you don't understand.
21. Ask Bob to walk you through testing. Bob will explain how to install dependencies, how to set up credentials, how to test the script locally. Follow the steps; ask questions when stuck.
22. Once it works, ask Bob to add error handling, logging, and a config file so you can change the recipient list without editing the code. Bob will refactor accordingly.
23. Save the workflow as a skill. "Create a skill called weekly-report that captures the structure of this script so I can do something similar next time without starting from scratch."

The persona running this workflow is not a developer. They cannot write Python from a blank file. But they can read code, describe what they want, follow steps, and recognize when something is wrong. With Bob, they ship a working tool. Without Bob, they probably don't.

What Bob is not (for this audience)

It's worth being explicit about what Bob does not provide for citizen developers, so expectations match reality.

- **There is no visual builder.** Everything happens through text, chat, files, code. Users who need to manipulate a UI by dragging components should look elsewhere.
- **There is no spreadsheet surface.** Bob does not let you build formulas, pivot tables, or charts through a spreadsheet-like UI. Excel and Google Sheets remain the right tools for that.
- **There is no app-store distribution.** When Bob helps you build a script or tool, you still need to figure out how to run it, share it, or deploy it. Bob can help with these steps but doesn't host or distribute artifacts.
- **There is no Bob mobile app for non-coders.** The IDE runs on macOS, Linux, and Windows. There is no tablet or phone-friendly low-code surface.
- **Code review and security review still need a human.** Bob will write code that works for non-traditional developers, but that code needs to be reviewed by someone qualified before it goes into production, especially if it touches sensitive data or external services.

How to use Bob as a citizen developer responsibly

For non-traditional developers, the highest-leverage habits are slightly different than for professional developers.

- Always start in Plan mode. The discipline of getting a plan before generating code is more important when you cannot easily evaluate the code yourself. The plan is something you can read and understand even if the code that implements it is opaque.
- Use Ask mode liberally. Before you change anything, explore. Ask Bob to explain what files exist, what they do, what would happen if you changed something. Ask mode is read-only, so you cannot accidentally break anything.
- Pair with a developer for review. Find a colleague who codes and ask them to review what Bob produced before you ship it. This is not optional for anything that touches production data or external services. Bob will write code that works in tests but might fail in unexpected ways in production, and you need a second opinion.
- Keep auto-approve off. The IDE's safety model exists for a reason. Approving each action gives you the chance to ask 'is this what I expected?' even if you cannot evaluate the code in detail.
- Use checkpoints. Bob's shadow Git repository (Part VI) means you can always restore to a previous state if something goes wrong. For non-traditional developers who cannot easily undo by inspecting changes, the checkpoint system is a particularly valuable safety net.
- Build a personal skill library. Every workflow that worked for you once is a candidate for a skill. Over time, your skill library becomes the way you express what you need without re-explaining the technical details every time.

The honest assessment is this: Bob is not a replacement for true no-code platforms when you need to build apps without writing any code at all. But Bob extends the reach of someone who can describe what they want and read the result, into territory that previously required a developer. For citizen developers in technical roles, analysts, devops-adjacent operations people, technical product managers, business technologists, Bob is a meaningful productivity tool that the rest of the no-code market doesn't quite touch.

Part XV: Mainframe and platform specifics, IBM Z and IBM i

Bob's mainframe and midrange stories are at different stages of maturity and worth covering separately. The Z story is real today through the Premium Package for Z, currently in technical preview. The IBM i story is partly real today and partly anticipated through the planned iMode capability. For organizations running these platforms, both are likely to be the most important parts of Bob's value proposition.

IBM Bob Premium Package for Z

Announced alongside general availability on April 28 2026 and currently available as a no-cost private technical preview, the Premium Package for Z integrates the capabilities of IBM watsonx Code Assistant for Z into Bob and adds mainframe-specific intelligence on top. The package is delivered as an extension to a Bob trial, requested through the IBM sales process at early-access.ibm.com.

Two new modes are the headline. Architect mode performs system-level reasoning across mainframe applications. It understands application structure, identifies dependencies, infers business intent, and assesses change impact before any updates are made. The mode combines application metadata with multi-model intelligence to deliver context-aware, enterprise-wide insights, the kind of analysis that mainframe modernization projects typically spend months producing by hand.

Code mode in the Z package is differentiated from base Bob's Code mode by Z-aware context. It generates, refactors, and transforms COBOL, PL/I, JCL, and related mainframe languages with awareness of platform-specific idioms. It supports editing, linting, and debugging for z/OS applications. It leverages rich contextual signals, application metadata, language and middleware awareness, platform-specific tooling, to produce code that fits the way mainframe systems actually work, rather than what a generic LLM might suggest based on training data.

The APIS IT case study is the most concrete public demonstration of what Bob does for mainframe modernization. APIS IT, the Croatian government IT agency, used Bob to modernize mission-critical systems spanning decades of technical debt, mainframe and .NET environments combined. The reported results: 10x faster architecture analysis and documentation for a twenty-year-old EGL/CIS system, 100 percent operator-verified accuracy in Croatian-language documentation of decades-old systems, and migration of complex .NET services in hours rather than weeks. For organizations facing similar modernization challenges, this is the proof-of-concept reference.

IBM i: the iMode anticipation

Bob can be used by IBM i developers today, but the experience is incomplete. Bob's standalone IDE works as a fork of VS Code, which means anything VS Code can do for IBM i development (through extensions like Code for IBM i) is, in principle, available through Bob-IDE, though IBM's

documentation does not officially endorse this path. Bob's language-level support for RPG, CL, and SQL exists, and the model routing layer includes capability for these languages.

What Bob cannot do today, but is planned to do through iMode, is the native IBM i integration that RPG developers expect. The planned capabilities, based on IT Jungle's coverage of Project Bob and direct community feedback to IBM, include direct work with source physical files (the traditional storage mechanism for RPG sources on IBM i), work with the IFS (integrated file system), library-list-aware compilation, awareness of fixed-form versus free-form RPG, integration with display files and external file definitions, and respect for IBM i call stacks and program object behavior. No release date is committed.

The IT Jungle independent review captures both the current state and the community's anticipation well. Gregory Simmons, an IBM Champion who tested an early-access build, was impressed by Bob's general capability, particularly its use of markdown files as living style guides and its ability to detect missing artifacts (it noticed a missing display file in his test RPG program and generated a working one without being asked). His honest assessment is that Bob is a real tool for IBM i developers today, but iMode is what would make it a category-defining one. The community is not waiting on Bob to fill the AI-for-RPG gap, many IBM i shops are already using GitHub Copilot with VS Code's Code for IBM i extension, but Bob with iMode is what would shift the conversation.

> *"The gap is not AI. The gap is AI that understands RPG, COBOL, SQL and CL and the workflows that come with building applications on this platform."*
> — Gregory Simmons, IBM Champion, writing in IT Jungle

Practical guidance for mainframe and midrange teams

For Z teams, the right move today is to engage with the technical preview of the Premium Package for Z. The preview is no-cost, the access process is straightforward (engage your IBM sales team, request access through the early-access portal, receive the extension), and the upside is direct exposure to a tool genuinely designed for mainframe modernization rather than a general-purpose coding assistant adapted for it. Existing WCA for Z customers have an adoption path.

For IBM i teams, the right move today is more nuanced. Bob-IDE can be used today for cloud-native parts of an IBM i environment, anything written in modern languages, anything that lives in Git rather than QSYS, anything that can be edited as a regular file. For the RPG-and-source-physical-file core of an IBM i shop, GitHub Copilot inside VS Code with the Code for IBM i extension remains the more mature option in early-to-mid 2026. The strategic choice is to engage with IBM about iMode timelines, participate in feedback channels, and be ready to switch when iMode ships. The expected payoff, Bob with native IBM i awareness, full SDLC orchestration across the platform's languages and storage models, would be significant if delivered.

For both audiences, the broader IBM positioning matters. Bob is the strategic AI-for-development product across IBM's platform portfolio. Premium Packages exist for Z. iMode is

planned for IBM i. The product strategy explicitly mentions further premium packages for additional IBM platforms in the future. Whether your team is on Z, IBM i, Power, or hybrid environments, Bob is the path IBM is committing to. The alternative would be staying on the WCA generation, which is in active maintenance but is not where the new investment is going.

Part XVI: Independent perspectives and the competitive landscape

IBM's marketing is unsurprisingly enthusiastic about Bob, and the case studies it publishes are picked to look good. A guide that takes only IBM's word for it would be incomplete. This chapter pulls together independent reviews, analyst commentary, journalist skepticism, and direct competitive context to give a more balanced view of where Bob actually stands.

The Omdia placement

The 2026 Omdia Universe report on AI-assisted software development named IBM Bob a Leader in the IDE-based tools category. The report expanded its evaluation criteria for the first time, weighting full-lifecycle capability (testing, security, deployment, orchestration) alongside coding. The shift was explicit and Omdia analyst Michael Azoff documented it in the report's framing: "The tools pulling ahead are the ones that handle testing, security, deployment, and orchestration. Not just code generation."

That framing favors Bob's design. Bob was built as an SDLC-spanning agent, not a coding assistant, which is the structural reason Bob's evaluation looks strong in a methodology that values lifecycle breadth. It also helps explain why a relatively young product (six months into general availability at the time of the Omdia evaluation) placed as a Leader against incumbents with significantly more history. Bob is being graded on a curve that matches its design choices.

The honest read of the Omdia placement is that it validates Bob's positioning rather than declaring it best-in-class against every measure. Other tools in the IDE-based category (GitLab Duo, GitHub Copilot, and Cursor) also scored well, and depending on which dimension matters most to a given team, the right choice may not be Bob. The Omdia placement is a strong signal that Bob is in the top tier of the market, not that it is the unambiguous winner.

The Bobathon experience

Northdoor, a UK IT services company with a thirty-year history with IBM, sent two developers to IBM's Bobathon in May 2026, a hands-on developer day where IBM invites teams to stress-test Bob against real coding problems. The Bobathon writeup is one of the most useful independent assessments of Bob because it covers actual development work, not demos.

The Northdoor developers, Reema Sharma and Dennis Baffour, built a complete application end-to-end during the Bobathon, requirements through deployment, including unit tests with MOQ and a CI/CD pipeline. Their assessment, captured in Northdoor's published writeup, was that Bob "held up" and in places surprised them. The two highlights were the breadth of coverage (the same tool spanned requirements through testing and CI, not just coding) and the security awareness (Bob identified and highlighted security flaws in a way the developers found genuinely valuable). The honest framing in the writeup is that the strongest argument for Bob over consumer-grade alternatives is security being secure by default at the point the code is written, rather than being added as a separate review step.

> *"Within a couple of hours, BOB was able to support and streamline the entire process very effectively, from requirements through to development and testing. It demonstrated how AI can significantly improve productivity, accelerate delivery, and help maintain code quality across the SDLC."*
> — Reema Sharma, Software Developer, Northdoor

The IT Jungle IBM i first-look

Gregory Simmons's IT Jungle review of the early-access Bob build for IBM i developers (covered partially in Part XI) is the most detailed independent review of Bob from a platform-specific perspective. Simmons is an IBM Champion with thirty years on the platform, exactly the kind of practitioner whose opinion carries weight in the IBM i community.

Simmons's review is enthusiastic but disciplined. He praises the standalone VS Code fork as the right architectural choice (drawing the contrast with WCA for i's extension approach), highlights Bob's markdown-based team style guides as "a linter on steroids," and uses the missing-DSPF anecdote, Bob noticed a display file was missing from his test program and generated a working one, as evidence of Bob's contextual awareness. He is equally clear that the current build is early. iMode for native IBM i support is planned but not delivered. Bob cannot work directly with source physical files yet. Many shops, including his own, are already using GitHub Copilot with the Code for IBM i extension and getting strong results.

The conclusion is one of the most quoted external assessments of Bob: "Bob is still early, but the foundation is promising. If iMode brings the features many of us are hoping for, Bob has a real chance to become one of the most important tools IBM has introduced for this platform in years." That is qualified optimism from a practitioner whose enthusiasm cannot be bought.

The skeptical view

Joe Fay's coverage in The Register on the day of Bob's general-availability launch is the most skeptical mainstream piece on the product. The piece raises three points worth engaging with seriously.

First, the 80,000 internal IBM users are not an unbiased sample. The Register notes that IBM is the latest in a line of companies, PwC, Accenture, to lean on its own staff to demonstrate the efficacy of an AI platform, with staff progression now tied to AI adoption. The 45 percent productivity claim is self-reported. The 70 percent gains on the Instana team and 69 percent gains on the Maximo team are self-reported. The numbers are not necessarily wrong, but they are not independently verified, and a guide to Bob owes you that context.

Second, GitHub recently reworked Copilot pricing because complex multi-step prompts were costing more than the subscription was bringing in. Bob's whole value proposition is multi-step agentic work, the exact category that broke Copilot's flat-rate model. The Bobcoin overage system is IBM's answer, but it means heavy users should expect their bills to scale with their usage in ways that flat-rate competitors hide.

Third, the brand. "Bob" inevitably evokes Microsoft Bob, the 1995 consumer Windows shell that is widely regarded as one of the worst products in tech history. The Register's tongue-in-cheek

subhead, "80,000 internal guinea pigs, Bobcoins, mainframe dreams and a name that really should have raised more flags", captures the press response. The naming is a real risk. Whether Bob the AI development partner can shed Bob the Microsoft mascot remains to be seen.

The Register also notes the January 2026 security disclosure (covered in Part VIII), that researchers found Bob could be manipulated into executing malware via the CLI, and that the IDE was vulnerable to common AI-specific data-exfiltration vectors. These are presumably addressed in GA, but they are part of the public record.

The competitive landscape

Bob does not exist in a vacuum. The AI-coding-tools market in 2026 is crowded, and a developer evaluation should be in the context of the alternatives.

Tool	Surface	Models	Primary positioning
IBM Bob	Standalone IDE (VS Code fork) + CLI	Claude, Mistral, Granite, specialists (routed)	AI-first SDLC partner with enterprise governance
GitHub Copilot	Extension across editors + chat	Mix selected per task (not published)	Autocomplete-first, expanding into agents
Anthropic Claude Code	CLI + various editor integrations	Anthropic Claude only	Agentic terminal for engineering teams
Cursor	Standalone IDE (VS Code fork) + agent	User picks (OpenAI, Anthropic, others)	AI-first IDE with deep editor integration
OpenAI Codex / ChatGPT in IDE	CLI + various integrations	OpenAI models	Agentic coding from OpenAI's ecosystem
GitLab Duo	Integrated into GitLab	Selected by GitLab	AI built into the DevOps platform

A few patterns are worth naming. Standalone IDEs (Bob, Cursor) are the form factor most willing to commit to a different developer experience; extensions (Copilot, Codex in editors) prioritize fitting into existing workflows. Multi-model routing is unique to Bob in mainstream tools, though Copilot does something similar without exposing the logic. Built-in sandboxing at the agent level is differentiated for Bob's CLI. Pricing models vary widely: flat subscriptions (Copilot, Cursor), API-based pass-through (Claude Code), credit-based metering (Bob).

The honest competitive read is that Bob is best-positioned for enterprises that value governance, lifecycle breadth, multi-model flexibility, and IBM portfolio integration, particularly anyone on Z or planning to be on iMode for IBM i. Bob is less well-positioned for individual developers who want maximum model control, for shops already deep in the GitHub or GitLab

ecosystem, or for cost-sensitive solo use where flat-rate competitors offer better economics. The right tool depends on the team.

Closing, what to do next

If you've read this far, you have a working model of what Bob is, where it came from, what it does, what it doesn't do, and how it stacks up against the alternatives. The question now is what to do about it.

For individual developers, the right next step is the free trial. IBM gives you 40 Bobcoins for thirty days, enough to complete the Node.js quickstart from Part III, work through a substantive modernization task, evaluate Bob against your daily workflow, and decide whether the Pro/Pro+/Ultra economics work for you. The trial is at bob.ibm.com/trial. Sign up, install Bob-IDE, complete the quickstart, then try Bob on a real task you would have done anyway. Compare the result honestly to how you would have done it without Bob. If the answer is meaningfully better, the subscription pays for itself.

For engineering teams, the right next step is a structured pilot. Pick a small group, five to ten developers across different roles, and put them on Pro+ for a month. Pick three categories of work to evaluate: green-field development, modernization or refactoring, and reviews or testing. Document the time spent and outcomes in each category. Compare against your existing baselines. At the end of the month you'll have direct evidence for whether Bob belongs in your toolchain and which tier makes sense at organizational scale.

For engineering leaders evaluating Bob as an organizational decision, the right next step is engaging with IBM sales for an enterprise conversation. The Enterprise tier is qualitatively different from the individual plans in ways that matter for organizational rollouts, shared Bobcoin pooling, centralized administration, SSO, priority support, custom Bobcoin pack purchasing. The economics also become more favorable at scale, with per-Bobcoin pricing dropping as volume increases. The right conversation is not 'should we buy Bob' but 'if we deployed Bob across our engineering organization, what would the structure and economics look like.'

For organizations on IBM Z, the technical preview of the Premium Package for Z is the highest-leverage thing you can do today. The preview is no-cost, the value proposition is platform-specific in ways the base Bob offering cannot match, and engaging now puts you in conversation with IBM as the product matures. For organizations on IBM i, the right move is engagement with IBM about iMode, what it will include, when it will ship, what input IBM is taking from practitioners, combined with using Bob's general-purpose capabilities for the parts of your IBM i environment that aren't bound to source physical files today.

Beyond any of these specific next steps, Bob is worth understanding even if you decide not to adopt it. The bet Bob represents, that the next phase of AI in software development is full-lifecycle orchestration rather than coding speed alone, is increasingly the consensus position among analysts and major vendors. Whether you end up on Bob, on a competitor that adopts a similar model, or on a tool that wins by being differently designed, the framing of what good AI-assisted development looks like in 2026 has shifted. This guide is meant to leave you with the framing and the facts, ready to act on either.

If you decide to ship something based on what you learned here, an internal evaluation, a team adoption plan, an organizational pilot, a published review of your own, the goal of this guide will have been met. Bob is a meaningful product. The developer community deserves an honest, complete, working map of what it actually is. That's what this has tried to be.

— End —

Anil Singh

Based on publicly available information current to May 2026

Sources: bob.ibm.com documentation, IBM newsroom, IBM Think 2026, Omdia Universe 2026, IT Jungle, Northdoor, AIBusiness, The Register, VentureBeat, RedMonk analysis

Appendix A: The Galaxium Travels tutorial path

The Node.js modernization quickstart described in Part III is one of two official learning paths IBM publishes for Bob. The other is a structured eight-tutorial sequence built around the Galaxium Travels demo application, a fictional luxury space travel booking system. Where the quickstart is breadth-first (one Bob session that touches modes, mentions, approvals, and literate coding), the Galaxium path is depth-first, each tutorial isolates a single capability and explores it thoroughly. For developers who want to internalize Bob systematically rather than learn by doing, the Galaxium path is the more rigorous route.

This appendix gives the orientation map: what each tutorial covers, what it teaches, and roughly how long it takes. It is a companion to the official tutorials, not a replacement for them. The full step-by-step content, complete with code listings, exercise specifications, and validation steps, lives at bob.ibm.com/docs/ide/getting-started/tutorials. The repository is at github.com/IBM/galaxium-travels, with the bob-learning-path-branch checked out by default through the clone command in the documentation. Read this appendix to decide which tutorial to invest time in next; follow IBM's documentation to actually do the work.

The Galaxium Travels application

The demo application is a full-stack booking system with a React/TypeScript front end, a Python back end exposing both REST and MCP interfaces, and a SQLite database with seed data. The front end supports viewing flights, user registration and authentication, and flight booking. The back end has data models for flights, users, and bookings, plus schemas for validation.

Two interface choices in Galaxium Travels are worth noticing because they're pedagogically deliberate. First, the back end exposes both a REST API and an MCP interface, the MCP interface lets Bob interact with the back end as a tool, which is the closest thing the official tutorials get to demonstrating MCP in action against a non-trivial service. Second, an API browser is provided so you can test endpoints directly without the front end, which makes it easy to verify Bob's changes without spinning up the full stack.

The eight tutorials, in order

The tutorials are designed to build on each other. The official sequence is:

#	Tutorial	What you learn
1	Create application features (introduction)	Architecture orientation. Sets up the Galaxium repo, explains the front-end / back-end / database layout, and frames the eight-tutorial arc.
2	Start a project with /init and AGENTS.md	How Bob's stateless nature is addressed by persistent context files. Generates five AGENTS.md files, one root and four mode-specific, and shows the maintenance cadence for keeping them current.

#	Tutorial	What you learn
3	Standardize Bob's behavior	Project-level rules files. Codifies team coding standards as files Bob automatically applies on every interaction. Introduces the rule-vs-mode distinction (rules are cross-cutting; mode instructions are mode-scoped).
4	Add Bob capabilities	Building a custom product-manager mode. Walks through every field in the mode definition with concrete example values, then tests the resulting mode by asking it to recommend the next feature to build.
5	Get code predictions	Next Edit experimental completion mode. Shows the diff-oriented inline-completion experience and how it differs from traditional autocomplete.
6	Generate code from comments	Literate coding mode. Writing instructions in plain English directly inside source files and having Bob generate the code in place.
7	Plan and implement complex features	Pairing with Bob across multiple modes and tools for a non-trivial feature. The closest the official tutorials get to a real production workflow.
8	Create a new context window	Context window management. When to start a new chat, how condensation works, how to prevent context poisoning, and how MCP loadouts and large files affect Bobcoin economics.

The combined time to complete all eight tutorials is roughly four to six hours for a developer comfortable with React, Python, and Git. For a developer who is new to one of those stacks, expect closer to eight or ten, most of the time is reading and validating Bob's output, not waiting for it.

Where the Galaxium path fits in your Bob ramp

Three usage patterns are worth knowing if you're planning a learning sequence.

For a solo developer ramping up on Bob, the recommended order is the Quickstart first (thirty to sixty minutes, real work, end-to-end feel), then dip into the Galaxium tutorials individually as you encounter the features in your daily work. You do not need to do them in sequence, tutorial six on literate coding makes sense the first time you want to use literate coding, not before.

For a team standardizing on Bob, the right model is to assign the Galaxium tutorials as onboarding material. Each developer completes the full eight-tutorial sequence in their first week with Bob. The team's collective fluency comes out of shared completion of the same

scaffolding. Tutorials three (rules) and four (custom modes) are particularly important here because they are the tutorials that produce shared team artifacts.

For an engineering leader evaluating Bob for organizational adoption, the right model is to have one or two senior engineers complete the full Galaxium path before any procurement conversation. The tutorials are designed to surface the strengths and limitations of Bob honestly, by the end, those engineers will have informed opinions, which is exactly what you want for a procurement decision.

Appendix B: AGENTS.md and /init in depth

The /init command and the AGENTS.md files it generates are described briefly throughout this guide, but the mechanics warrant a dedicated reference. /init is the single highest-leverage one-time investment per project, and understanding what it actually creates is the difference between using Bob effectively and re-establishing context every conversation.

Why /init exists

Large language models are stateless. Each new conversation starts with no memory of previous interactions. Without explicit project context, Bob would have to discover the codebase from scratch every time you opened a new chat, reading files, inferring structure, guessing conventions. At scale, this is impractical: every conversation would burn Bobcoins on the same discovery work.

/init solves the problem by generating persistent context files that Bob loads automatically on every future conversation. The files describe the project, its structure, technology stack, conventions, workflows, in a format optimized for an AI agent's consumption. They function, in IBM's own framing, like onboarding documentation for Bob.

What /init actually generates

Running /init in Code mode produces five files, not one. This is a detail that's easy to miss but matters for understanding how Bob uses context.

File	Location	Purpose
`AGENTS.md`	Workspace root	The main project context. Loaded on every conversation regardless of mode. Covers project overview, directory structure, technology stack, architectural patterns, development workflows.
`AGENTS-code.md`	`.bob/rules-code/`	Code mode-specific context. Emphasizes file structure, naming patterns, and conventions that matter when writing or modifying code.
`AGENTS-plan.md`	`.bob/rules-plan/`	Plan mode-specific context. Emphasizes architectural conventions, system boundaries, and design patterns that matter when planning implementations.
`AGENTS-ask.md`	`.bob/rules-ask/`	Ask mode-specific context. Emphasizes the kind of information a developer is likely to ask about, APIs, behaviors, common pitfalls.

File	Location	Purpose
`AGENTS-advanced.md`	`.bob/rules-advanced/`	Advanced mode-specific context. Combines aspects of all the above for the unrestricted-tool-access mode.

The mode-specific files extend the main AGENTS.md rather than replacing it. When you start a conversation in Code mode, Bob loads the main AGENTS.md plus AGENTS-code.md. When you switch to Plan mode, Bob loads the main AGENTS.md plus AGENTS-plan.md. This layered structure means you can have shared cross-cutting context (in the root file) and mode-tailored context (in the mode-specific files) without duplication.

What goes in a good AGENTS.md

/init's auto-generated AGENTS.md is a strong starting point but not the end state. The official guidance is to edit AGENTS.md manually to add context that Bob's automated scan might not capture, business rules, deployment conventions, team-specific practices, the things that live in a senior engineer's head rather than in the code.

A well-maintained AGENTS.md typically contains:

- **Project overview and purpose.** What this codebase exists to do, who uses it, what the success metrics are. Two or three paragraphs at most.
- **Directory structure and key file locations.** Where each layer lives, what the main entry points are, where configuration lives.
- **Technology stack and dependencies.** Languages, frameworks, key libraries, runtime requirements. Versions matter, Bob will write code that targets whatever versions the file mentions.
- **Architectural patterns and conventions.** How things are structured, MVC versus hexagonal, event-driven versus request-response, monolith versus microservices. Whatever the project's actual pattern is.
- **Development workflows.** How to build, test, deploy. What CI runs. What the branching model is.
- **Business rules.** Things you would tell a new engineer on day one that aren't in the code. Domain invariants, regulatory constraints, team-specific quirks.

When to re-run /init

The AGENTS.md files are not auto-updated as the project changes. Bob does not watch your files and rewrite the context as you commit. You need to re-run /init manually when major changes happen. The documentation recommends re-running after any of:

24. Adding new modules or services that change the project's top-level structure
25. Changing the directory structure (renames, reorganizations, moves)

26. Adopting new technologies or frameworks, major version bumps, new languages, new build tools
27. Onboarding new team members who will use Bob (re-run to make sure the context is current)

Re-running /init regenerates the files. Manual edits you've made to AGENTS.md will be lost unless you protect them. The recommended workflow is to keep a separate notes file with your team-specific additions, re-run /init when needed, then re-apply your additions. Some teams use a comment block at the top of AGENTS.md ("-- MANUAL ADDITIONS BELOW THIS LINE --") and a discipline of restoring that block after each /init.

AGENTS.md and the broader convention

AGENTS.md is not a Bob invention. The convention was pioneered by Cursor and adopted by Aider and other AI-first development tools, and Bob honors it deliberately. If you have an AGENTS.md file in your workspace root from another tool, Bob loads it by default. This makes it possible to switch between tools without rebuilding team standards.

The bob-code.useAgentRules setting controls this behavior. By default it's true, Bob respects AGENTS.md. If you want full control through Bob's own files instead, you can disable it, but doing so loses portability. Most teams should leave it enabled.

The relationship between AGENTS.md, rules, and modes

A common point of confusion is how AGENTS.md, custom rules, and custom modes interact. They are different mechanisms with overlapping uses, and getting the distinction right matters for designing your Bob configuration.

Mechanism	Scope	Typical use
AGENTS.md	Project-wide, all modes	Persistent project context: what is this codebase, how is it structured. Generated by /init, refined manually.
Rules (.bobrules, .bob/rules/)	Project-wide, all modes (or mode-scoped if in .bob/rules-{slug}/)	Cross-cutting behavior standards: coding style, commenting conventions, testing requirements. Loaded automatically into every conversation.
Mode customInstructions	Single mode only	Mode-specific tactical guidance: when in Plan mode do X, when in security-reviewer mode do Y. Lives inside the mode definition.

The clean separation is: AGENTS.md describes the project, rules describe how to work, modes describe what role you're playing right now. A well-configured project has all three, with each capturing what it captures best and nothing else.

Appendix C: Enterprise deployment reference

This appendix consolidates everything an engineering organization needs to know to deploy Bob across a team or organization. It is reference material, not a sequential walkthrough, and assumes you have already decided to adopt Bob and are now planning the rollout.

Network requirements

Bob is a SaaS application hosted in IBM's us-east region. It cannot operate offline, and it cannot be self-hosted as of mid-2026, IBM has signaled an on-prem release for data-residency and regulated workloads but no firm date. Your network must allow outbound HTTPS (TCP port 443) traffic to the IBM domains used by Bob.

The official firewall allowlist for Bob is documented at bob.ibm.com/docs/ide/troubleshooting/ts-firewall-rules. The eight domains that must be reachable are:

Domain	Purpose
`*.us-east.bob.ibm.com`	Bob service endpoints (wildcard covers subdomain routing)
`api.us-east.bob.ibm.com`	Bob API endpoint for tool execution and model inference
`iam.cloud.ibm.com`	IBM Cloud Identity and Access Management for authentication and authorization
`console-ibm-prod.verify.ibm.com`	IBM identity verification services
`idaas.ice.ibmcloud.com`	IBM Cloud identity and access services
`www.ibm.com`	IBM corporate domain for product information and resources
`login.ibm.com`	IBMid authentication endpoint
`myibm.ibm.com`	MyIBM portal for managing IBM SaaS subscriptions

For organizations behind a proxy, all eight domains need to be reachable through the proxy. For organizations with strict egress policies, this list is the minimum, you may also want to allow Bob's update endpoints if you plan to keep the IDE current automatically. The us-east hosting is worth flagging for European customers with strict data-localization requirements; the on-prem option, when it ships, will be the path for those workloads.

Identity and authentication

Bob uses IBMid for individual authentication and supports enterprise SSO through your existing identity provider on the Enterprise tier. For individual Pro/Pro+/Ultra users, IBMid signup is free

at ibm.com/account/reg. For Enterprise deployments, the right model is SSO integration during the initial activation step, so users do not need separate IBMid accounts.

The authentication flow is OAuth-based. On first launch of Bob-IDE or first invocation of BobShell, the browser opens for sign-in. The resulting session token is stored locally and remains valid until you sign out explicitly or the session is revoked centrally. For BobShell in CI/CD pipelines, API-key authentication is supported through the --auth-method api-key flag, API keys are managed through the bob.ibm.com account settings or through the admin dashboard for Enterprise plans.

The admin dashboard

Enterprise plans include access to the Administration page at bob.ibm.com/admin, which provides centralized control over the organization's Bob deployment. The dashboard surfaces five capabilities.

Capability	What it does
Shared Bobcoin pool	Distribute Bobcoins from a centralized organizational budget to team members rather than maintaining per-user allocations. Real-time reallocation between teams.
User management (Seats)	Add, remove, and manage team member access. Seats can be reassigned between individuals as team membership changes.
Spending controls	Set individual spending limits per user. Adjust based on team needs without restructuring the entire allocation.
Usage insights	Monitor Bobcoin consumption across users, teams, and time. Identify high-cost patterns and optimize resource allocation.
Team organization	Group users into teams with role assignments. Push instant updates that take effect without user action.

Enterprise economics: Bobcoin packs

The Enterprise tier replaces per-user monthly allocations with a shared organizational Bobcoin pool. The pool is replenished through Bobcoin packs purchased through your IBM sales representative.

The published pack pricing is straightforward. Each pack contains 1,000 Bobcoins. Each pack costs $500 USD. Organizations purchase as many packs as needed. There is no per-user cost on top of the packs in the Enterprise model, the entire cost is the Bobcoin spend.

> **Important: Bobcoin packs expire**
> Bobcoins from Enterprise packs expire one year after the purchase date. This is different

> from the Pro/Pro+/Ultra individual plans, where monthly allocations reset at the start of each billing cycle.
>
> The practical implication is that organizations should size their pack purchases to actual annual usage rather than overbuying for future capacity. Bobcoins purchased today that aren't consumed within twelve months are forfeited.
>
> For organizations with seasonal usage patterns (e.g., heavy modernization sprints in Q1, lighter ongoing development the rest of the year), the right model is multiple smaller pack purchases timed to usage rather than one large upfront purchase.

To put the math in perspective, a 1,000-Bobcoin pack at $500 yields a per-Bobcoin cost of $0.50, the same as the published consumer rate. Volume discounts at the Enterprise tier are negotiated with sales and depend on commitment level. For an organization with 100 active developers each consuming roughly 100 Bobcoins per month, annual consumption would be approximately 120,000 Bobcoins or 120 packs ($60,000), with the understanding that any unused balance expires.

Account activation and team setup

The Enterprise activation flow proceeds in a documented sequence. The detailed steps live at bob.ibm.com/docs/ide/enterprise/getting-started, but the structural arc is consistent across deployments.

28. Activate the instance. IBM provisions the organization in their identity system; an administrator logs into bob.ibm.com/admin for the first time and completes the activation flow.
29. Configure SSO. The administrator connects the organization's identity provider through standard SAML or OIDC integration.
30. Create teams. Organize users into teams that reflect organizational structure (engineering, security, data, platform, etc.) Each team can have independent Bobcoin allocation limits.
31. Add users. Invite users by email or import from the identity provider. Users sign in through SSO using their existing organizational credentials.
32. Allocate Bobcoins. Distribute the available pool across teams based on expected usage. Adjustments are real-time and require no user action.
33. Distribute organizational artifacts. Push the team's custom modes, rules, and AGENTS.md templates so new users start with the right configuration.

The recommended pilot pattern, validated by Anthropic's experience with internal AI tooling and the Bob customer references, is to start with five to ten developers across different roles for thirty days, measure outcomes against your existing baselines, then scale to broader rollout based on the evidence. The Pro+ tier ($60/user/month with 160 Bobcoins) is a reasonable approximation of the per-user economics during the pilot phase; Enterprise economics become more favorable at scale.

Appendix D: Troubleshooting reference

Most Bob installations work without intervention. When they don't, the documented troubleshooting topics fall into four categories: connection issues, installation issues, layout issues, and last-resort resets. This appendix captures each.

Connection and authentication issues

The most common cause of Bob being unable to connect is a corporate firewall blocking outbound traffic to IBM's domains. Symptoms include connection errors and timeouts when attempting to start a Bob session, sign-in failures during the IBMid OAuth flow, or model inference timeouts mid-session.

The solution is to coordinate with your network administrator to add the eight Bob domains documented in Appendix C to your firewall's allowlist. HTTPS on TCP port 443 is the only protocol/port required. After the firewall change is applied, restart Bob, the new rules will not take effect for the existing process.

If the firewall is correctly configured but Bob still cannot connect, the next thing to check is proxy configuration. Bob honors standard HTTP_PROXY, HTTPS_PROXY, and NO_PROXY environment variables. If your organization uses a transparent proxy with TLS inspection, you may need to add the proxy's CA certificate to your system trust store; otherwise Bob's TLS validation will fail.

Slow performance after installing or updating

If Bob feels noticeably slow after a fresh install or update, the most common cause is installing the wrong package for your architecture, particularly on Apple Silicon (arm64) Macs that accidentally installed the Intel (x86_64) build. Bob will run in Rosetta emulation but at significantly degraded performance.

To verify you have the correct build on macOS: open the Bob application, then in the Activity Monitor look for the Bob process. The Kind column shows either Apple or Intel. If it shows Intel on an Apple Silicon machine, you have the wrong build. Download the correct architecture from bob.ibm.com/download and reinstall.

On Windows and Linux the equivalent check is whether the installed package matches your CPU architecture. The .deb and .rpm packages are typically architecture-specific in their filenames (e.g., ibm-bob_amd64.deb versus ibm-bob_arm64.deb).

Interface and panel position issues

The Bob chat panel sometimes appears in unexpected locations after updates or workspace changes, particularly if you've imported settings from VS Code, Cursor, or Windsurf that had their AI panels in different positions. The panel can disappear from its expected sidebar position, appear in a secondary editor area, or in extreme cases become hidden entirely.

To reposition the Bob panel, right-click on its title bar and select "Move to" with the desired position (primary sidebar, secondary sidebar, panel, editor). Alternatively, use the View menu's panel management commands. The position persists across workspace switches once explicitly set.

If the panel has disappeared entirely, the View → Bob menu option (or Cmd+L / Ctrl+L) brings it back. If keyboard shortcuts don't restore it, the last resort is the full reset described below.

Resetting Bob to resolve persistent issues

When standard troubleshooting steps fail and Bob is in a state where normal operation is impossible, corrupted settings, persistent crashes, broken authentication, the documented last resort is a full reset of the application state. This is destructive: any unsaved customizations are lost, and you will need to re-authenticate.

The reset procedure varies by platform but follows the same arc: quit Bob completely, remove the application's user data directory (the location is platform-specific and documented at bob.ibm.com/docs/ide/troubleshooting/ts-reset-application), restart Bob, sign in again. Custom modes and rules that live in your workspace's .bob/ directory are preserved because they're part of the workspace; only the application's own state is reset.

> **Before doing a full reset**
>
> Export your current settings if possible, Bob's settings JSON can be copied and restored after the reset.
>
> Note any custom keyboard shortcuts you've configured.
>
> Identify any extensions you've installed; you'll need to re-add them after reset.
>
> Custom modes and rules in .bob/ inside your workspace are workspace artifacts and survive the reset. Custom modes and rules in ~/.bob/ (global) do not, back them up before resetting.

When Bob made changes you didn't want

The most stress-inducing situation in agentic AI usage is when the agent does something you didn't want, multiple steps deep into a session, and you need to undo it. Bob's answer to this is checkpoints (Part VI), and the recommended workflow is: scroll up in the chat to locate the checkpoint just before the unwanted change, click "Restore Checkpoint," choose "Restore Files and Task," and confirm. This rolls back both the workspace files and the conversation context, putting you back in the state before the problem started.

Checkpoint restore is more reliable than trying to fix things forward with corrective prompts. The documentation is explicit: if Bob is producing subpar work mid-session, restore is the right answer, not better prompting. Context contamination compounds, and restore is the only documented way to clear it.

For changes that fell outside Bob's checkpoint coverage (manual edits, changes to ignored files), the safety net is Git. If you've been committing frequently throughout the session, git reset or git checkout returns you to the pre-change state. The combination of Bob checkpoints for AI-mediated changes and Git for manual changes covers nearly every reasonable recovery scenario.

Appendix E: The canonical auto-approval risk matrix

Part VI describes the auto-approval toolbar and its nine action categories with Claude's interpretation of relative risk. This appendix gives IBM's canonical published risk matrix from the official security guidelines at bob.ibm.com/docs/ide/security/bob-security-guidance, which is shorter than Claude's version (four categories rather than nine) and worth knowing because it represents IBM's official position on which auto-approvals are most dangerous.

IBM's published risk levels

Setting	Risk level	IBM's recommendation
Edit files	High	Enable only in controlled environments
Execute commands	High	Use whitelist and avoid wildcards
Use MCP servers	Medium-High	Only with trusted servers
Read files	Medium	Consider sensitive data exposure

The framing IBM uses in the official documentation is worth quoting directly: "Auto-approve settings bypass confirmation prompts, giving Bob direct access to your system. This can result in data loss, file corruption, or worse. Command line access is particularly dangerous, as it can potentially run harmful operations."

This is unusually direct language for vendor security documentation. Most AI tool vendors describe auto-approval as a productivity feature; IBM describes it as a feature that can result in data loss or worse. The choice of language is a deliberate signal about how IBM expects developers to think about it.

Practical implementation guidance

Combining the IBM risk matrix with the operational guidance from Part VI yields a working policy that most teams can adopt directly.

Read auto-approval (Medium risk) is acceptable for most projects after the first week. Bob reading files is genuinely low-risk in the sense that it cannot modify anything, and manual approval for every file read is tedious enough to discourage exploration. Enable it once you trust Bob's discretion about which files to read.

Edit auto-approval (High risk) should be off for any production-adjacent project. The risk of Bob modifying the wrong file or making an unwanted change is too high. The Greg Hintermeister observation noted in Part XVI, Bob removing functionality from the running app but leaving the underlying code intact, is exactly the kind of subtle wrong-file edit that auto-approve would hide. Keep this manual.

Execute auto-approval (High risk) should be off by default. If you must enable it, do so only with a tight allowlist of safe commands: git status, npm test, ls, cat, and the specific build/test commands for your project. The two-stage approval model documented in Part VI is the right pattern, auto-approval enabled at the toolbar level, individual commands added to the allowed list through explicit approval the first time they run. Never put rm, git push --force, or anything touching production credentials on an allowlist.

MCP auto-approval (Medium-High risk) depends entirely on which MCP servers you've configured. Local STDIO servers running code you've reviewed are relatively safe; remote SaaS MCP integrations to systems you didn't build are less so. Treat MCP auto-approval as a per-server decision rather than a global one, Bob lets you set alwaysAllow patterns at the server level for exactly this reason.

> *"Always review Bob's output to ensure it is accurate and that any generated code will act as intended. Never inherently trust output from any AI system."*
> — IBM Bob Security Guidelines

Appendix F: Bobcoin economics in depth

Part XIII covered pricing at the level needed for evaluation. This appendix is the reference-grade treatment, drawing directly from the official Bobcoins documentation at bob.ibm.com/docs/ide/account/bobcoins. It applies primarily to individual plans (Pro, Pro+, Ultra). The Enterprise pack economics are covered in Appendix C.

The canonical definition

Bobcoins are the consumption-based billing metric used for Bob subscriptions. The IBM-published framing: "Bobcoins are resource units that measure your consumption of Bob's AI capabilities. Each action you complete with Bob, such as generating code, analyzing files, or running commands, consumes a certain number of Bobcoins based on the computational resources required."

The key characteristics IBM publishes:

Characteristic	Description
Monthly reset	Your Bobcoin allocation resets at the start of each billing cycle (individual plans). Enterprise packs do not reset, they expire one year after purchase.
Transparent pricing	One Bobcoin equals $0.50 USD. This is the single conversion rate that applies across all plans.
Pay-as-you-go overage	You only pay for what you use beyond your base allocation. Overage is metered in Bobcoins at the same conversion rate.
Predictable billing	Clear visibility into usage and costs through the Bobcoin gauge, Settings page, and Bobalytics.

Bobcoins versus tokens, the official explanation

The relationship between Bobcoins and the underlying token economics of the language models is described directly in IBM's documentation. The official explanation: "Tokens are the raw units of model usage. Tokens represent pieces of text (words, characters) that the AI model processes. Every request to the model consumes input tokens (your prompt and context) and output tokens (Bob's response). Bobcoins are IBM's billing metric that abstracts token usage into a predictable cost structure. One Bobcoin represents a standardized amount of computational resources, which includes token usage across different models and operations."

The practical implication: you do not need to track tokens directly. Bob automatically converts your model usage into Bobcoins for billing. Different models and operations consume tokens at different rates, but Bobcoins give you a single consistent unit to predict your costs against.

This abstraction is a feature, not a limitation. With API-based pricing on competitor tools, developers and engineering managers need to understand input vs output token pricing, model-specific multipliers, context-window growth effects, and the different rates between premium and cheaper models. With Bobcoins, you watch one number. The trade-off is reduced granularity, you can't see exactly which underlying model handled which task, but for budgeting purposes the abstraction is favorable.

Monitoring usage: three surfaces

Bob exposes Bobcoin consumption through three surfaces, each suited to a different kind of monitoring.

Surface	Where	Best for
Gauge component	Top-right corner of the Bob panel	Real-time awareness during sessions. Hover for current usage against monthly allocation.
Settings page	Settings → General tab	Detailed account information and usage statistics. The right surface for understanding where you stand mid-month.
Bobalytics dashboard	Bobalytics feature (Part VI)	Usage patterns over time, feature-level consumption, team-level visibility for Enterprise plans.

Base allocations by plan

The official table of monthly Bobcoin allocations:

Plan	Monthly Bobcoins included	Implied monthly value
Free trial	40 Bobcoins (30 days)	$20 USD equivalent
Pro	40 Bobcoins	$20 USD equivalent, base subscription is $20
Pro+	160 Bobcoins	$80 USD equivalent, base subscription is $60
Ultra	500 Bobcoins	$250 USD equivalent, base subscription is $200

The pattern is straightforward: as you move up the plans, the effective per-Bobcoin rate drops because the base allocation grows faster than the subscription price. Ultra at $200 for 500 Bobcoins is effectively $0.40 per Bobcoin, 20 percent off the published rate. Pro+ at $60 for 160

Bobcoins is effectively $0.375 per Bobcoin, 25 percent off. Volume discounting is baked into the plan structure.

Practical budgeting

Translating Bobcoin allocations into practical capacity requires understanding which kinds of work consume what. Without published per-action consumption rates from IBM, the guidance is empirical, track your own consumption for the first month and project from there.

Patterns that emerge from the customer stories and community reports: lightweight uses (small completions, single-mode Q&A, brief literate-coding edits) extend an allocation a long way. Heavy uses (large modernization tasks like the Blue Pearl Java 21 upgrade, multi-file refactors with extensive context, Orchestrator-mode workflows coordinating multiple specialized modes) consume Bobcoins quickly. The Blue Pearl team spent three hours up front constructing context, that context-setting alone likely consumed a significant chunk of their Bobcoin budget for the project, even before any modernization happened.

A reasonable rule of thumb for individual developers: a Pro subscription (40 Bobcoins/month) covers light daily assistance and one or two substantive tasks per month. Pro+ (160 Bobcoins) covers most professional development workflows with comfortable headroom. Ultra (500 Bobcoins) covers heavy modernization or systems work without forcing you to think about consumption. Below those thresholds, you'll hit overage. Above them, you have headroom for unexpected complex work.

Appendix G: Glossary and acronym key

Bob's documentation uses a mixture of Bob-specific terminology, IBM portfolio terms, AI-industry standard terms, and conventions inherited from other AI development tools. This glossary defines every term used in this guide in alphabetical order. Where a term has multiple meanings, the Bob-specific meaning is given.

Term	Definition
Advanced mode	Built-in Bob mode with full unrestricted tool access (read, edit, browser, command, MCP). Same role definition as Code mode but with extended capabilities. For complex workflows requiring all tool groups.
AGENTS convention	Industry convention pioneered by Cursor and adopted by Bob, Aider, and other tools: a markdown file at the workspace root that AI tools load as persistent project context.
AGENTS.md	Persistent project-context file generated by /init. Loaded automatically by Bob on every conversation. Convention shared with Cursor, Aider, and other AI-first development tools. Bob also generates four mode-specific AGENTS-{mode}.md files in .bob/rules-{mode}/.
Architect mode	Mainframe-specific mode in the IBM Bob Premium Package for Z. Performs system-level reasoning across mainframe applications, identifying dependencies and assessing change impact.
Ask mode	Built-in Bob mode with read-only file access plus browser and MCP. Cannot modify code. For explanations, exploration, and analysis without risk.
Auto-approve toolbar	UI element in Bob-IDE's chat panel that controls which Bob actions can run without explicit per-action approval. Has nine action categories: Read, Write, Browser, MCP, Mode, Subtasks, Execute, Question, Todo.
Bob Findings	Centralized panel for findings produced by Bob tips and code reviews. Each finding has severity, location, status (Open/In Progress/Resolved), and Fix-with-Bob action.
Bob tips	Bob feature that runs continuously in the background, analyzing open files for code-quality issues. Marks problematic functions with purple underlines and surfaces findings in the Bob Findings panel.
Bob-IDE	Bob's standalone development environment. A fork of Visual Studio Code with IBM-developed AI capabilities baked into the editor rather than layered as an extension.
.bob/	Workspace-root directory containing project-scoped Bob configuration: custom modes (custom_modes.yaml), rules (rules/, rules-{mode}/), skills (skills/), MCP server config (mcp.json), and slash commands (commands/).

Term	Definition
Bobalytics	Bob feature that surfaces usage analytics: Bobcoin consumption over time, feature-level usage patterns, team-level visibility on Enterprise plans.
Bobcoin	Bob's consumption-based billing unit. One Bobcoin equals $0.50 USD. Abstracts underlying token economics into a single unit users can budget against.
Bobcoin pack	Enterprise-only purchase unit: 1,000 Bobcoins for $500 USD. Expires one year after purchase date. The replenishment mechanism for shared Enterprise Bobcoin pools.
.bobignore	Workspace-root file using .gitignore syntax that specifies files and directories Bob's tools cannot read or write. Actively monitored, changes apply immediately. Does not create a system-level sandbox.
.bobrules	Optional workspace-root file containing project-wide custom rules. The simpler alternative to .bob/rules/ directory layout for projects that don't need mode-specific rules.
BobShell	Bob's terminal agent. Separate command-line tool installed independently of Bob-IDE. Supports interactive (bob) and non-interactive (bob -p) modes. Includes built-in sandboxing via macOS Seatbelt and Docker/Podman.
Checkpoint	Automatic snapshot of workspace state taken before every Bob file modification. Implemented as a shadow Git repository separate from the project's actual Git history. Used for undoing unwanted Bob changes.
Code mode	Built-in Bob mode with read, edit, and command tool access. Default mode for implementation work. Highly skilled software engineer persona.
Context mention	Reference to a specific project element in a Bob prompt, prefixed with @. Seven types: file path, folder path, @problems, @terminal, @<commit-hash>, @git-changes, @<URL>.
Context poisoning	Failure mode where bad data enters the conversation context and progressively degrades output quality. Documented as not fixable through corrective prompts, only resolution is starting a new chat.
Context window	Bob's working memory for a conversation. 200,000 tokens maximum, with automatic condensation at 140,000 tokens.
Custom mode	User-defined Bob persona with custom role definition, instructions, tool group access, and optional file regex constraints. Defined in YAML at .bob/custom_modes.yaml (project) or custom_modes.yaml in user directory (global).

Term	Definition
Custom rule	User-defined behavioral constraint loaded into context on every Bob interaction. Lives in .bobrules, .bobrules-{mode}, .bob/rules/, or .bob/rules-{mode}/. Cross-cutting; applies regardless of mode.
DSPF	Display file. IBM i UI definition file used by RPG programs to define screen layouts. Bob can generate matching DSPF files when an RPG program references screen logic without an existing DSPF.
Enhance prompt	Bob feature that refines a vague user prompt into a structured one with explicit requirements. The sparkles icon at the bottom right of the chat input.
Enterprise tier	Bob's organizational plan. Shared Bobcoin pool, SSO integration, admin dashboard, priority support, custom pack purchasing. Pricing through IBM sales.
Galaxium Travels	IBM's official demo application for Bob tutorials. Fictional luxury space travel booking system at github.com/IBM/galaxium-travels. Full stack: React/TypeScript front end, Python back end with REST and MCP interfaces, SQLite database.
Granite	IBM's family of small language models. One of the model sources Bob's routing layer selects from, used primarily for lightweight inline tasks and low-latency work.
IBM i Developer Mode	Mainframe-specific mode planned for the IBM Bob Premium Package for i (scheduled June 24, 2026). Configured for native IBM i languages: RPG, CL, SQL/DB2 for i, COBOL, DDS.
IBMid	IBM's identity service. The default authentication method for Bob individual plans. Free signup at ibm.com/account/reg.
iMode	Internal IBM communications name for the planned IBM i awareness capability. Now shipping as IBM i Developer Mode in the Premium Package for i.
/init	Built-in slash command that scans a project and generates five AGENTS.md files: one root file and four mode-specific files. Establishes persistent project context.
Inline edit	Cmd+K / Ctrl+K shortcut that triggers a quick Bob edit at the cursor position without opening the full chat panel.
Instana	IBM observability platform. One of the named internal IBM teams that piloted Bob, reporting 70 percent reduction in time spent on selected tasks.

Term	Definition
Literate coding	Bob mode where instructions are written in plain language directly inside source files. Bob generates the code in place. Toggle with Cmd+M / Ctrl+M.
Maximo	IBM asset management platform. One of the named internal IBM teams that piloted Bob, reporting 69 percent time saving on code generation and refactoring.
MCP (Model Context Protocol)	Open standard for connecting AI tools to external services. Bob supports MCP at both project (.bob/mcp.json) and global (mcp_settings.json) levels. Three transports: STDIO, Streamable HTTP, SSE.
Mistral	Open-source language model family. One of the model sources Bob's routing layer selects from, used primarily for cost-efficient general coding work.
Next Edit	Experimental Bob completion mode. Diff-oriented inline completions with gutter indicators and jump decorations. Configurable via Completion Mode dropdown in settings.
Orchestrator mode	Built-in Bob mode with no direct tool access. Coordinates complex multi-step projects by delegating subtasks to specialized modes. Workflow coordinator persona.
Plan mode	Built-in Bob mode with read access and edit access constrained to markdown files. Cannot modify source code. For architecture, design, breakdown work that produces planning documents.
Premium Package for i	IBM Bob extension for IBM i / RPG development. Scheduled for June 24, 2026 release. Will include IBM i Developer Mode persona and approximately 40 curated IBM i skills.
Premium Package for Z	IBM Bob extension for mainframe development. Currently in technical preview. Adds Architect mode and Z-aware Code mode for COBOL, PL/I, JCL.
Project Bob	Internal name for Bob during the private beta phase (October 2025 to April 2026).
RPG / RPGLE / SQLRPGLE	Report Program Generator. Programming language native to IBM i. RPGLE is the free-form variant; SQLRPGLE embeds SQL statements in RPG source.
Seatbelt	macOS sandboxing mechanism used by BobShell. Five profiles: permissive-open (default), permissive-closed, permissive-proxied, restrictive-open, restrictive-closed.

Term	Definition
Shadow Git repository	Hidden Git repository used by Bob's checkpoint system. Separate from the project's actual Git history. Snapshots workspace state before every file modification.
Skill	Reusable Bob workflow defined as a markdown file with YAML frontmatter. Lives in .bob/skills/ (project) or ~/.bob/skills/ (global). Available in Advanced mode.
SKILL.md	Conventional filename for a Bob skill definition. Same format used by Anthropic's Claude skills system, providing portability.
Slash command	Single-keystroke chat workflow trigger. Built-ins: /init, /review, /create-pr, plus mode-switch commands. Custom commands defined as markdown files in .bob/commands/ or ~/.bob/commands/.
SLM (Small Language Model)	Compact language models, in Bob's case primarily IBM Granite. Used by the routing layer for latency-sensitive or simple-completion tasks.
SSE (Server-Sent Events)	Legacy MCP transport for remote servers. Kept for backward compatibility. Streamable HTTP is the recommended remote transport for new development.
STDIO transport	MCP transport that runs the server locally as a child process of Bob, communicating through stdin/stdout. Fastest and most secure transport.
Streamable HTTP transport	Modern MCP remote transport. Single HTTP endpoint supporting both request-response and Server-Sent Events streaming.
Telemetry	Optional usage data collection. Bob's posture is opt-in (off by default). Categories: task openings, completion acceptance, scan results, conversation messages, tool invocations.
Two-stage approval	Approval pattern for the Execute action: first enable Execute auto-approval at the toolbar, then individual commands must be added to an allowed list before they run.
WCA (Watsonx Code Assistant)	Predecessor product to Bob. Existed in three variants: WCA general, WCA for Z (mainframe), WCA for i (IBM i / RPG). Bob is the rebrand and architectural rebuild.
--yolo	BobShell flag that enables write access in non-interactive mode. Write access is scoped to the directory where Bob was started, a hard guarantee at the tool layer.

Bobcoins versus tokens, the official explanation

The relationship between Bobcoins and the underlying token economics of the language models is described directly in IBM's documentation. The official explanation: "Tokens are the raw units of model usage. Tokens represent pieces of text (words, characters) that the AI model processes. Every request to the model consumes input tokens (your prompt and context) and output tokens (Bob's response). Bobcoins are IBM's billing metric that abstracts token usage into a predictable cost structure. One Bobcoin represents a standardized amount of computational resources, which includes token usage across different models and operations."

The practical implication: you do not need to track tokens directly. Bob automatically converts your model usage into Bobcoins for billing. Different models and operations consume tokens at different rates, but Bobcoins give you a single consistent unit to predict your costs against.

This abstraction is a feature, not a limitation. With API-based pricing on competitor tools, developers and engineering managers need to understand input vs output token pricing, model-specific multipliers, context-window growth effects, and the different rates between premium and cheaper models. With Bobcoins, you watch one number. The trade-off is reduced granularity, you can't see exactly which underlying model handled which task, but for budgeting purposes the abstraction is favorable.

Monitoring usage: three surfaces

Bob exposes Bobcoin consumption through three surfaces, each suited to a different kind of monitoring.

Surface	Where	Best for
Gauge component	Top-right corner of the Bob panel	Real-time awareness during sessions. Hover for current usage against monthly allocation.
Settings page	Settings → General tab	Detailed account information and usage statistics. The right surface for understanding where you stand mid-month.
Bobalytics dashboard	Bobalytics feature (Part VI)	Usage patterns over time, feature-level consumption, team-level visibility for Enterprise plans.

Base allocations by plan

The official table of monthly Bobcoin allocations:

Plan	Monthly Bobcoins included	Implied monthly value
Free trial	40 Bobcoins (30 days)	$20 USD equivalent

Plan	Monthly Bobcoins included	Implied monthly value
Pro	40 Bobcoins	$20 USD equivalent, base subscription is $20
Pro+	160 Bobcoins	$80 USD equivalent, base subscription is $60
Ultra	500 Bobcoins	$250 USD equivalent, base subscription is $200

The pattern is straightforward: as you move up the plans, the effective per-Bobcoin rate drops because the base allocation grows faster than the subscription price. Ultra at $200 for 500 Bobcoins is effectively $0.40 per Bobcoin, 20 percent off the published rate. Pro+ at $60 for 160 Bobcoins is effectively $0.375 per Bobcoin, 25 percent off. Volume discounting is baked into the plan structure.

Practical budgeting

Translating Bobcoin allocations into practical capacity requires understanding which kinds of work consume what. Without published per-action consumption rates from IBM, the guidance is empirical, track your own consumption for the first month and project from there.

Patterns that emerge from the customer stories and community reports: lightweight uses (small completions, single-mode Q&A, brief literate-coding edits) extend an allocation a long way. Heavy uses (large modernization tasks like the Blue Pearl Java 21 upgrade, multi-file refactors with extensive context, Orchestrator-mode workflows coordinating multiple specialized modes) consume Bobcoins quickly. The Blue Pearl team spent three hours up front constructing context, that context-setting alone likely consumed a significant chunk of their Bobcoin budget for the project, even before any modernization happened.

A reasonable rule of thumb for individual developers: a Pro subscription (40 Bobcoins/month) covers light daily assistance and one or two substantive tasks per month. Pro+ (160 Bobcoins) covers most professional development workflows with comfortable headroom. Ultra (500 Bobcoins) covers heavy modernization or systems work without forcing you to think about consumption. Below those thresholds, you'll hit overage. Above them, you have headroom for unexpected complex work.

Appendix G: Glossary and acronym key

Bob's documentation uses a mixture of Bob-specific terminology, IBM portfolio terms, AI-industry standard terms, and conventions inherited from other AI development tools. This glossary defines every term used in this guide in alphabetical order. Where a term has multiple meanings, the Bob-specific meaning is given.

Term	Definition
.bobignore	Workspace-root file using .gitignore syntax that specifies files and directories Bob's tools cannot read or write. Actively monitored, changes apply immediately. Does not create a system-level sandbox.
.bob/	Workspace-root directory containing project-scoped Bob configuration: custom modes (custom_modes.yaml), rules (rules/, rules-{mode}/), skills (skills/), MCP server config (mcp.json), and slash commands (commands/).
.bobrules	Optional workspace-root file containing project-wide custom rules. The simpler alternative to .bob/rules/ directory layout for projects that don't need mode-specific rules.
Advanced mode	Built-in Bob mode with full unrestricted tool access (read, edit, browser, command, MCP). Same role definition as Code mode but with extended capabilities. For complex workflows requiring all tool groups.
AGENTS.md	Persistent project-context file generated by /init. Loaded automatically by Bob on every conversation. Convention shared with Cursor, Aider, and other AI-first development tools. Bob also generates four mode-specific AGENTS-{mode}.md files in .bob/rules-{mode}/.
Architect mode	Mainframe-specific mode in the IBM Bob Premium Package for Z. Performs system-level reasoning across mainframe applications, identifying dependencies and assessing change impact.
Ask mode	Built-in Bob mode with read-only file access plus browser and MCP. Cannot modify code. For explanations, exploration, and analysis without risk.
AGENTS convention	Industry convention pioneered by Cursor and adopted by Bob, Aider, and other tools: a markdown file at the workspace root that AI tools load as persistent project context.
Auto-approve toolbar	UI element in Bob-IDE's chat panel that controls which Bob actions can run without explicit per-action approval. Has nine action categories: Read, Write, Browser, MCP, Mode, Subtasks, Execute, Question, Todo.
Bobalytics	Bob feature that surfaces usage analytics: Bobcoin consumption over time, feature-level usage patterns, team-level visibility on Enterprise plans.

Term	Definition
Bobcoin	Bob's consumption-based billing unit. One Bobcoin equals $0.50 USD. Abstracts underlying token economics into a single unit users can budget against.
Bobcoin pack	Enterprise-only purchase unit: 1,000 Bobcoins for $500 USD. Expires one year after purchase date. The replenishment mechanism for shared Enterprise Bobcoin pools.
Bob Findings	Centralized panel for findings produced by Bob tips and code reviews. Each finding has severity, location, status (Open/In Progress/Resolved), and Fix-with-Bob action.
Bob-IDE	Bob's standalone development environment. A fork of Visual Studio Code with IBM-developed AI capabilities baked into the editor rather than layered as an extension.
Bob tips	Bob feature that runs continuously in the background, analyzing open files for code-quality issues. Marks problematic functions with purple underlines and surfaces findings in the Bob Findings panel.
BobShell	Bob's terminal agent. Separate command-line tool installed independently of Bob-IDE. Supports interactive (bob) and non-interactive (bob -p) modes. Includes built-in sandboxing via macOS Seatbelt and Docker/Podman.
Checkpoint	Automatic snapshot of workspace state taken before every Bob file modification. Implemented as a shadow Git repository separate from the project's actual Git history. Used for undoing unwanted Bob changes.
Code mode	Built-in Bob mode with read, edit, and command tool access. Default mode for implementation work. Highly skilled software engineer persona.
Context mention	Reference to a specific project element in a Bob prompt, prefixed with @. Seven types: file path, folder path, @problems, @terminal, @<commit-hash>, @git-changes, @<URL>.
Context poisoning	Failure mode where bad data enters the conversation context and progressively degrades output quality. Documented as not fixable through corrective prompts, only resolution is starting a new chat.
Context window	Bob's working memory for a conversation. 200,000 tokens maximum, with automatic condensation at 140,000 tokens.
Custom mode	User-defined Bob persona with custom role definition, instructions, tool group access, and optional file regex constraints. Defined in YAML at .bob/custom_modes.yaml (project) or custom_modes.yaml in user directory (global).

Term	Definition
Custom rule	User-defined behavioral constraint loaded into context on every Bob interaction. Lives in .bobrules, .bobrules-{mode}, .bob/rules/, or .bob/rules-{mode}/. Cross-cutting; applies regardless of mode.
DSPF	Display file. IBM i UI definition file used by RPG programs to define screen layouts. Bob can generate matching DSPF files when an RPG program references screen logic without an existing DSPF.
Enhance prompt	Bob feature that refines a vague user prompt into a structured one with explicit requirements. The sparkles icon at the bottom right of the chat input.
Enterprise tier	Bob's organizational plan. Shared Bobcoin pool, SSO integration, admin dashboard, priority support, custom pack purchasing. Pricing through IBM sales.
Galaxium Travels	IBM's official demo application for Bob tutorials. Fictional luxury space travel booking system at github.com/IBM/galaxium-travels. Full stack: React/TypeScript front end, Python back end with REST and MCP interfaces, SQLite database.
Granite	IBM's family of small language models. One of the model sources Bob's routing layer selects from, used primarily for lightweight inline tasks and low-latency work.
IBMid	IBM's identity service. The default authentication method for Bob individual plans. Free signup at ibm.com/account/reg.
IBM i Developer Mode	Mainframe-specific mode planned for the IBM Bob Premium Package for i (scheduled June 24, 2026). Configured for native IBM i languages: RPG, CL, SQL/DB2 for i, COBOL, DDS.
iMode	Internal IBM communications name for the planned IBM i awareness capability. Now shipping as IBM i Developer Mode in the Premium Package for i.
/init	Built-in slash command that scans a project and generates five AGENTS.md files: one root file and four mode-specific files. Establishes persistent project context.
Inline edit	Cmd+K / Ctrl+K shortcut that triggers a quick Bob edit at the cursor position without opening the full chat panel.
Instana	IBM observability platform. One of the named internal IBM teams that piloted Bob, reporting 70 percent reduction in time spent on selected tasks.

Term	Definition
Literate coding	Bob mode where instructions are written in plain language directly inside source files. Bob generates the code in place. Toggle with Cmd+M / Ctrl+M.
Maximo	IBM asset management platform. One of the named internal IBM teams that piloted Bob, reporting 69 percent time saving on code generation and refactoring.
MCP (Model Context Protocol)	Open standard for connecting AI tools to external services. Bob supports MCP at both project (.bob/mcp.json) and global (mcp_settings.json) levels. Three transports: STDIO, Streamable HTTP, SSE.
Mistral	Open-source language model family. One of the model sources Bob's routing layer selects from, used primarily for cost-efficient general coding work.
Next Edit	Experimental Bob completion mode. Diff-oriented inline completions with gutter indicators and jump decorations. Configurable via Completion Mode dropdown in settings.
Orchestrator mode	Built-in Bob mode with no direct tool access. Coordinates complex multi-step projects by delegating subtasks to specialized modes. Workflow coordinator persona.
Plan mode	Built-in Bob mode with read access and edit access constrained to markdown files. Cannot modify source code. For architecture, design, breakdown work that produces planning documents.
Premium Package for Z	IBM Bob extension for mainframe development. Currently in technical preview. Adds Architect mode and Z-aware Code mode for COBOL, PL/I, JCL.
Premium Package for i	IBM Bob extension for IBM i / RPG development. Scheduled for June 24, 2026 release. Will include IBM i Developer Mode persona and approximately 40 curated IBM i skills.
Project Bob	Internal name for Bob during the private beta phase (October 2025 to April 2026).
RPG / RPGLE / SQLRPGLE	Report Program Generator. Programming language native to IBM i. RPGLE is the free-form variant; SQLRPGLE embeds SQL statements in RPG source.
Seatbelt	macOS sandboxing mechanism used by BobShell. Five profiles: permissive-open (default), permissive-closed, permissive-proxied, restrictive-open, restrictive-closed.

Term	Definition
Shadow Git repository	Hidden Git repository used by Bob's checkpoint system. Separate from the project's actual Git history. Snapshots workspace state before every file modification.
Skill	Reusable Bob workflow defined as a markdown file with YAML frontmatter. Lives in .bob/skills/ (project) or ~/.bob/skills/ (global). Available in Advanced mode.
SKILL.md	Conventional filename for a Bob skill definition. Same format used by Anthropic's Claude skills system, providing portability.
Slash command	Single-keystroke chat workflow trigger. Built-ins: /init, /review, /create-pr, plus mode-switch commands. Custom commands defined as markdown files in .bob/commands/ or ~/.bob/commands/.
SLM (Small Language Model)	Compact language models, in Bob's case primarily IBM Granite. Used by the routing layer for latency-sensitive or simple-completion tasks.
SSE (Server-Sent Events)	Legacy MCP transport for remote servers. Kept for backward compatibility. Streamable HTTP is the recommended remote transport for new development.
STDIO transport	MCP transport that runs the server locally as a child process of Bob, communicating through stdin/stdout. Fastest and most secure transport.
Streamable HTTP transport	Modern MCP remote transport. Single HTTP endpoint supporting both request-response and Server-Sent Events streaming.
Telemetry	Optional usage data collection. Bob's posture is opt-in (off by default). Categories: task openings, completion acceptance, scan results, conversation messages, tool invocations.
Two-stage approval	Approval pattern for the Execute action: first enable Execute auto-approval at the toolbar, then individual commands must be added to an allowed list before they run.
WCA (Watsonx Code Assistant)	Predecessor product to Bob. Existed in three variants: WCA general, WCA for Z (mainframe), WCA for i (IBM i / RPG). Bob is the rebrand and architectural rebuild.
--yolo	BobShell flag that enables write access in non-interactive mode. Write access is scoped to the directory where Bob was started, a hard guarantee at the tool layer.

This glossary covers the terminology used in this guide. The IBM documentation at bob.ibm.com/docs uses additional terminology for specific subsystems that fall outside the scope of this reference; consult the official documentation for those.

A note on completeness

This guide attempts to be comprehensive for a working developer or engineering leader. Comprehensive is not the same as complete. IBM publishes its documentation as living material, updated continuously, and any printed snapshot will lag the source. The most reliable practice is to use this guide for orientation and durable understanding, and to consult bob.ibm.com/docs for the current state of any specific feature, command flag, or pricing detail.

Where this guide has had to make choices, what to include, what to leave for the official docs, which independent reviews to weight, how to characterize honest limitations, the bias has been toward what a working developer actually needs to know rather than what would make the marketing read well. Where IBM's framing and independent reporting disagree, both are included. Where Bob has limitations, those are surfaced. The intent is a guide that a working engineer would trust enough to recommend to a colleague, and a guide that an engineering leader would trust enough to inform a procurement decision.

Both audiences deserve honesty about what comes next. Bob is a young product, less than a year out of general availability at the time of writing. The Premium Package for Z is in technical preview. The Premium Package for i ships in late June 2026. The on-prem option for regulated and data-residency workloads is signaled but not committed. The Bobcoin pricing model is being tested at scale for the first time. Independent benchmarks will accumulate over the next year. The honest picture two years from now may be different from the picture today, and the right posture for any reader is to keep reading the official documentation, the independent reviews, the customer stories, and the YouTube channel, rather than relying on any single source including this one.

The goal of this guide is to leave you well-equipped to do that ongoing reading. The framing, the vocabulary, the structure, the proof points, the honest limitations: these are durable, and they will help you make sense of new material as it comes. Use them well.

— End of guide —

ABOUT THIS GUIDE

The independent reference for IBM’s AI development partner.

Generally available since April 2026, IBM Bob is the most ambitious AI coding tool yet shipped at enterprise scale. It is not just an editor extension — it is a standalone IDE, a terminal agent, a multi-model routing layer, and a governance fabric that wraps every action.

This guide is the complete, evenhanded developer’s reference. It covers what Bob is, how to install it, how to configure it for a team, how its commands and modes actually work, and where the honest limitations remain. IBM’s strengths are surfaced; the gaps are not hidden. Independent reviews, customer stories, and analyst voices are integrated alongside the official documentation.

INSIDE

- Architecture, ecosystem, and the multi-model thesis
- Hands-on quickstart and Galaxium Travels tutorial path
- Complete commands reference (slash, keyboard, CLI, settings)
- Custom modes deep dive with three production-quality examples
- Twelve use cases drawn from public customer stories
- Pricing, Bobcoins, and enterprise deployment specifics
- Security guidelines, sandboxing, and honest disclosures
- Seven appendices covering deployment, troubleshooting, glossary

WHO THIS IS FOR

Developers shipping with Bob this week. Citizen developers exploring what it can do. Engineering leaders deciding whether it belongs in the toolchain.

Anil Singh

MAY 2026